RAJALAKSHMI ENGINEERING COLLEGE

RAJALAKSHMI NAGAR, THANDALAM – 602 105

RAJALAKSHMI
ENGINEERING COLLEGE

CD23621
MOBILE APPLICATION DESIGN AND DEVELOPMENT LABORATORY

Laboratory Observation Note Book

Name : Mohamed Zaidh A

Year / Branch / Section : 2026 / B.E Computer Science and Design / A

Register No. : 231701032

Semester : 6

Academic Year : 2025 - 2026

List of Experiments

1. Develop a mobile application that adapts to different screen sizes and orientations using Constraint Layout.

   Focus on aesthetic design, typography, and responsive UI. Handling different data formats. Design Focus: Layout adaptability, visual hierarchy.

2. Design a scientific calculator with custom UI components. The app should handle user input and offer visual feedback for button presses. Design Focus: Interaction design, visual feedback, aesthetics.

3. Create wireframes and a functional prototype for a mobile app using Figma. Convert the prototype into a basic Android application. Design Focus: Prototyping, flow consistency.

4. Implement gesture-based interactions such as swiping, pinch-to-zoom, and double-tap to navigate different screens or trigger actions in the app. Design Focus: Gesture interaction, smooth transitions.

5. Design and develop a mobile app that displays user data (e.g., steps, expenses) using custom-designed charts and graphs. Design Focus: Data visualization, clarity.

6. Design the UI for a chat application with message bubbles, avatars, and a responsive input field. Design Focus: Seamless interaction, UI consistency.

7. Create a mobile app that incorporates custom animations for user actions, such as button clicks or transitions between screens. Design Focus: Animation, user feedback.

8. Develop a location-based app that displays the user's current location on a map with interactive features such as markers and routes. Design Focus: Location-based UI, map interactivity.

9. Design custom notifications and alerts, integrating both visual and auditory feedback, and aligning the notification design with the app's theme. Design Focus: Notification design, feedback.

10. Create an app that captures images, applies filters, crops, and edits photos with a visually appealing UI. Design Focus: Image editing tools, intuitive interface.

11. Design and develop a complete mobile application, integrating multiple concepts learned during the course. Design Focus: End-to-end design, creativity, functional app architecture.

INDEX

Reg. No. : 231701032 Name : Mohamed Zaidh A

Year : 2026 Branch : B.E Computer Science and Design Sec : A

Ex. No. : 1 **Date: 21.01.26**

Register No.: 231701032 **Name: Mohamed Zaidh A**

Adaptive UI Design

Aim

Develop a mobile application that adapts to different screen sizes and orientations using Constraint Layout. Focus on aesthetic design, typography, and responsive UI. Handling different data formats. Design Focus: Layout adaptability, visual hierarchy.

Procedure

1. **Requirement Analysis**
   Identify the objective of designing an adaptive mobile UI that supports multiple screen sizes (small, medium, large devices) and different orientations (portrait and landscape). Analyze the layout adaptability and visual hierarchy requirements.
2. **Create a New Android Studio Project**
   - Open **Android Studio**.
   - Select **New Project → Empty Activity**.
   - Choose **Kotlin/Java** as the programming language.
   - Set the Minimum SDK as required (API 21 or above recommended).
3. **Design the Base Layout using ConstraintLayout**
   - Open `activity_main.xml`.
   - Set the root layout as **ConstraintLayout**.
   - Add UI components such as:
     - TextView (Title/Header)
     - ImageView (Optional banner or icon)
     - EditText / RecyclerView / CardView
     - Button elements
   - Apply constraints (Top, Bottom, Start, End) properly to avoid fixed positioning.
4. **Apply Responsive Design Principles**
   - Use `wrap_content` and `match_constraint (0dp)` instead of fixed dimensions.
   - Utilize **Guidelines** and **Barriers** in ConstraintLayout for better alignment.
   - Maintain proper margins and padding for spacing consistency.

5. **Create Alternative Layout for Landscape Mode**
    - Right-click `res` → Create `layout-land` resource folder.
    - Copy `activity_main.xml` into `layout-land`.
    - Redesign the layout to arrange components horizontally for landscape view.
6. **Support Different Screen Sizes**
    - Create resource folders such as:
        - `layout-sw600dp` (for tablets)
    - Adjust UI elements (e.g., increase text size, multi-column layout).
7. **Typography and Styling**
    - Define text sizes in `res/values/dimens.xml`.
    - Use `styles.xml` to maintain consistent theme and typography.
    - Maintain visual hierarchy using:
        - Large bold font for headings
        - Medium font for subheadings
        - Standard size for body text
8. **Handle Different Data Formats**
    - Display text, images, and formatted numeric values.
    - Use string resources (`strings.xml`) instead of hardcoding values.
    - Ensure proper formatting using placeholders if needed.
9. **Testing the Application**
    - Run the app in Emulator.
    - Test on different virtual devices (Phone, Tablet).
    - Rotate the device to verify adaptive behavior.
    - Ensure no overlapping or distortion of UI elements.
10. **Optimization and Refinement**

- Check alignment warnings in Layout Inspector.
- Remove hardcoded dimensions.
- Improve spacing and consistency.

11. **Build and Run the Application**

- Clean and Rebuild the project.
- Execute the app and verify functionality.

AndroidManifest.xml

```
<?xml version="1.0" encoding="utf-8"?>
<manifest xmlns:android="http://schemas.android.com/apk/res/android"
    xmlns:tools="http://schemas.android.com/tools">

    <application
        android:allowBackup="true"
        android:dataExtractionRules="@xml/data_extraction_rules"
        android:fullBackupContent="@xml/backup_rules"
        android:icon="@mipmap/ic_launcher"
        android:label="@string/app_name"
        android:roundIcon="@mipmap/ic_launcher_round"
        android:supportsRtl="true"
        android:theme="@style/Theme.Ex1"
        tools:targetApi="31">
        <activity
            android:name=".MainActivity"
            android:exported="true">
            <intent-filter>
                <action android:name="android.intent.action.MAIN" />

                <category android:name="android.intent.category.LAUNCHER" />
            </intent-filter>
        </activity>
    </application>

</manifest>
```

activity_main.xml

```
<?xml version="1.0" encoding="utf-8"?>
<androidx.constraintlayout.widget.ConstraintLayout
    xmlns:android="http://schemas.android.com/apk/res/android"
    xmlns:app="http://schemas.android.com/apk/res-auto"
    android:id="@+id/rootLayout"
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    android:padding="16dp">

    <TextView
        android:id="@+id/tvTitle"
        android:layout_width="0dp"
        android:layout_height="wrap_content"
        android:text="Adaptive UI Design"
        android:textStyle="bold"
        android:textSize="@dimen/title_text"
        android:textAlignment="center"
        app:layout_constraintTop_toTopOf="parent"
        app:layout_constraintStart_toStartOf="parent"
        app:layout_constraintEnd_toEndOf="parent"/>

    <TextView
        android:id="@+id/tvSubTitle"
        android:layout_width="0dp"
```

```
        android:layout_height="wrap_content"
        android:text="Responsive layout • Typography • Data Formats"
        android:textAlignment="center"
        android:textSize="@dimen/subtitle_text"
        android:layout_marginTop="6dp"
        app:layout_constraintTop_toBottomOf="@id/tvTitle"
        app:layout_constraintStart_toStartOf="parent"
        app:layout_constraintEnd_toEndOf="parent"/>

    <!-- Left Card -->
    <androidx.cardview.widget.CardView
        android:id="@+id/cardInfo"
        android:layout_width="0dp"
        android:layout_height="0dp"
        app:cardCornerRadius="16dp"
        app:cardElevation="6dp"
        android:layout_marginTop="14dp"
        app:layout_constraintTop_toBottomOf="@id/tvSubTitle"
        app:layout_constraintBottom_toBottomOf="parent"
        app:layout_constraintStart_toStartOf="parent"
        app:layout_constraintEnd_toStartOf="@id/btnChangeData"
        app:layout_constraintWidth_percent="0.55">

        <LinearLayout
            android:layout_width="match_parent"
            android:layout_height="match_parent"
            android:orientation="vertical"
            android:gravity="center"
            android:padding="16dp">

            <TextView
                android:id="@+id/tvCollege"
                android:layout_width="match_parent"
                android:layout_height="wrap_content"
                android:text="Rajalakshmi Engineering College"
                android:textStyle="bold"
                android:textSize="@dimen/body_text"
                android:textAlignment="center"/>

            <TextView
                android:id="@+id/tvAmount"
                android:layout_width="match_parent"
                android:layout_height="wrap_content"
                android:text="Amount: ₹0.00"
                android:textSize="@dimen/body_text"
                android:textAlignment="center"
                android:layout_marginTop="10dp"/>

            <TextView
                android:id="@+id/tvDate"
                android:layout_width="match_parent"
                android:layout_height="wrap_content"
                android:text="Date: --"
                android:textSize="@dimen/body_text"
                android:textAlignment="center"
                android:layout_marginTop="8dp"/>
```

```
        </LinearLayout>
    </androidx.cardview.widget.CardView>

    <!-- Right Buttons -->
    <Button
        android:id="@+id/btnChangeData"
        android:layout_width="0dp"
        android:layout_height="wrap_content"
        android:text="Change Data Format"
        android:layout_marginTop="14dp"
        app:layout_constraintTop_toBottomOf="@id/tvSubTitle"
        app:layout_constraintStart_toEndOf="@id/cardInfo"
        app:layout_constraintEnd_toEndOf="parent"
        app:layout_constraintWidth_percent="0.42"/>

    <Button
        android:id="@+id/btnChangeTheme"
        android:layout_width="0dp"
        android:layout_height="wrap_content"
        android:text="Toggle Theme Colors"
        android:layout_marginTop="10dp"
        app:layout_constraintTop_toBottomOf="@id/btnChangeData"
        app:layout_constraintStart_toStartOf="@id/btnChangeData"
        app:layout_constraintEnd_toEndOf="@id/btnChangeData"/>

    <TextView
        android:id="@+id/tvNote"
        android:layout_width="0dp"
        android:layout_height="wrap_content"
        android:text="Landscape layout adapts into two columns."
        android:textAlignment="center"
        android:textSize="@dimen/caption_text"
        android:layout_marginTop="10dp"
        app:layout_constraintTop_toBottomOf="@id/btnChangeTheme"
        app:layout_constraintStart_toStartOf="@id/btnChangeData"
        app:layout_constraintEnd_toEndOf="@id/btnChangeData"/>

</androidx.constraintlayout.widget.ConstraintLayout>
```

layout-land/activity_main.xml:

```
<?xml version="1.0" encoding="utf-8"?>
<androidx.constraintlayout.widget.ConstraintLayout
    xmlns:android="http://schemas.android.com/apk/res/android"
    xmlns:app="http://schemas.android.com/apk/res-auto"
    android:id="@+id/rootLayout"
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    android:padding="16dp">

    <!-- Header -->
    <TextView
        android:id="@+id/tvTitle"
        android:layout_width="0dp"
        android:layout_height="wrap_content"
```

```
        android:text="Adaptive UI Design"
        android:textAlignment="center"
        android:textStyle="bold"
        android:textSize="@dimen/title_text"
        app:layout_constraintTop_toTopOf="parent"
        app:layout_constraintStart_toStartOf="parent"
        app:layout_constraintEnd_toEndOf="parent"/>

    <!-- Subtitle -->
    <TextView
        android:id="@+id/tvSubTitle"
        android:layout_width="0dp"
        android:layout_height="wrap_content"
        android:text="Responsive layout • Typography • Data Formats"
        android:textAlignment="center"
        android:textSize="@dimen/subtitle_text"
        app:layout_constraintTop_toBottomOf="@id/tvTitle"
        app:layout_constraintStart_toStartOf="parent"
        app:layout_constraintEnd_toEndOf="parent"
        android:layout_marginTop="8dp"/>

    <!-- Card-like container -->
    <androidx.cardview.widget.CardView
        android:id="@+id/cardInfo"
        android:layout_width="0dp"
        android:layout_height="wrap_content"
        app:cardCornerRadius="16dp"
        app:cardElevation="6dp"
        android:layout_marginTop="16dp"
        app:layout_constraintTop_toBottomOf="@id/tvSubTitle"
        app:layout_constraintStart_toStartOf="parent"
        app:layout_constraintEnd_toEndOf="parent">

        <LinearLayout
            android:layout_width="match_parent"
            android:layout_height="wrap_content"
            android:orientation="vertical"
            android:padding="16dp">

            <TextView
                android:id="@+id/tvCollege"
                android:layout_width="match_parent"
                android:layout_height="wrap_content"
                android:text="Rajalakshmi Engineering College"
                android:textStyle="bold"
                android:textSize="@dimen/body_text"
                android:textAlignment="center"/>

            <TextView
                android:id="@+id/tvAmount"
                android:layout_width="match_parent"
                android:layout_height="wrap_content"
                android:text="Amount: ₹0.00"
                android:textSize="@dimen/body_text"
                android:textAlignment="center"
                android:layout_marginTop="8dp"/>
```

```
            <TextView
                android:id="@+id/tvDate"
                android:layout_width="match_parent"
                android:layout_height="wrap_content"
                android:text="Date: --"
                android:textSize="@dimen/body_text"
                android:textAlignment="center"
                android:layout_marginTop="6dp"/>

        </LinearLayout>
    </androidx.cardview.widget.CardView>

    <!-- Buttons -->
    <Button
        android:id="@+id/btnChangeData"
        android:layout_width="0dp"
        android:layout_height="wrap_content"
        android:text="Change Data Format"
        android:layout_marginTop="16dp"
        app:layout_constraintTop_toBottomOf="@id/cardInfo"
        app:layout_constraintStart_toStartOf="parent"
        app:layout_constraintEnd_toEndOf="parent"/>

    <Button
        android:id="@+id/btnChangeTheme"
        android:layout_width="0dp"
        android:layout_height="wrap_content"
        android:text="Toggle Theme Colors"
        android:layout_marginTop="10dp"
        app:layout_constraintTop_toBottomOf="@id/btnChangeData"
        app:layout_constraintStart_toStartOf="parent"
        app:layout_constraintEnd_toEndOf="parent"/>

    <!-- Bottom note -->
    <TextView
        android:id="@+id/tvNote"
        android:layout_width="0dp"
        android:layout_height="wrap_content"
        android:text="Rotate screen / try tablet emulator to see adaptability."
        android:textSize="@dimen/caption_text"
        android:textAlignment="center"
        android:layout_marginTop="12dp"
        app:layout_constraintTop_toBottomOf="@id/btnChangeTheme"
        app:layout_constraintStart_toStartOf="parent"
        app:layout_constraintEnd_toEndOf="parent"/>

</androidx.constraintlayout.widget.ConstraintLayout>
```

dimes.xml

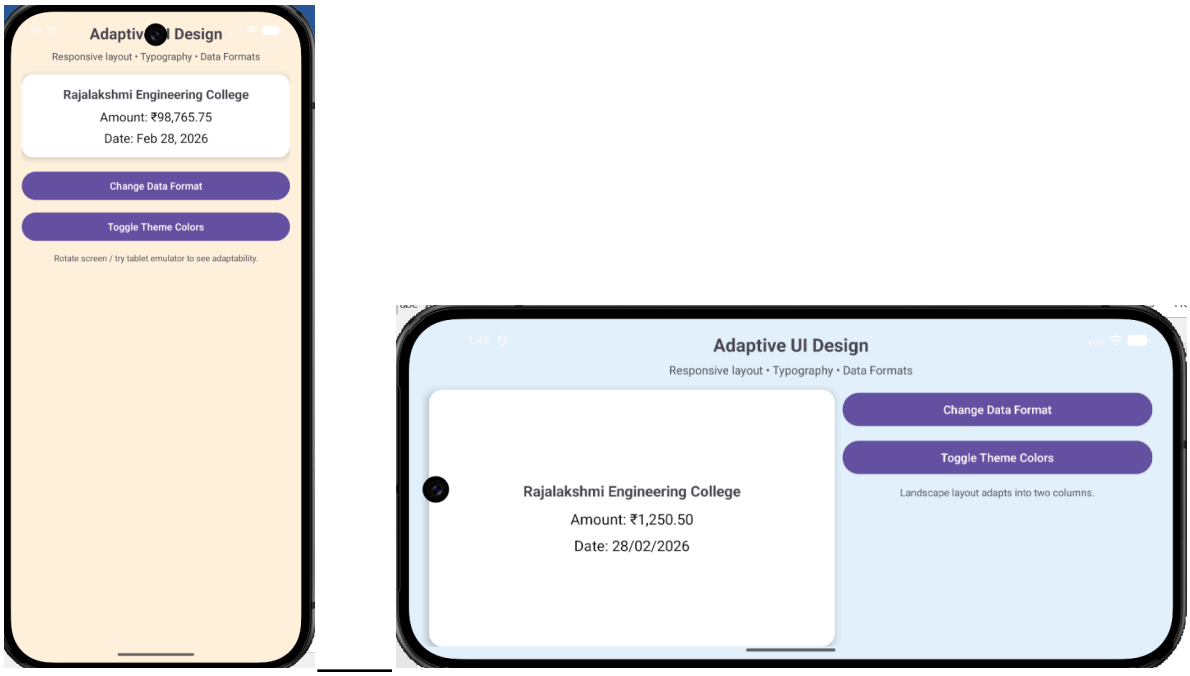
```
<?xml version="1.0" encoding="utf-8"?>
<resources>
    <dimen name="title_text">22sp</dimen>
    <dimen name="subtitle_text">14sp</dimen>
    <dimen name="body_text">18sp</dimen>
    <dimen name="caption_text">12sp</dimen>
</resources>
```

MainActivity.kt

```
package com.example.ex1

import android.graphics.Color
import android.os.Bundle
import android.widget.Button
import android.widget.TextView
import androidx.appcompat.app.AppCompatActivity
import androidx.constraintlayout.widget.ConstraintLayout
import java.text.NumberFormat
import java.text.SimpleDateFormat
import java.util.Date
import java.util.Locale

class MainActivity : AppCompatActivity() {

    private var themeIndex = 0
    private var dataIndex = 0

    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)
        setContentView(R.layout.activity_main)

        val rootLayout: ConstraintLayout = findViewById(R.id.rootLayout)
        val tvAmount: TextView = findViewById(R.id.tvAmount)
        val tvDate: TextView = findViewById(R.id.tvDate)

        val btnChangeData: Button = findViewById(R.id.btnChangeData)
        val btnChangeTheme: Button = findViewById(R.id.btnChangeTheme)

        // Initial values
        updateData(tvAmount, tvDate)

        btnChangeData.setOnClickListener {
            dataIndex++
            updateData(tvAmount, tvDate)
        }

        btnChangeTheme.setOnClickListener {
            themeIndex++
            when (themeIndex % 3) {
                0 -> {
                    rootLayout.setBackgroundColor(Color.WHITE)
                    tvAmount.setTextColor(Color.DKGRAY)
```

```
                    tvDate.setTextColor(Color.DKGRAY)
                }
                1 -> {
                    rootLayout.setBackgroundColor(Color.parseColor("#FFF3E0"))
// light orange
                    tvAmount.setTextColor(Color.BLACK)
                    tvDate.setTextColor(Color.BLACK)
                }
                2 -> {
                    rootLayout.setBackgroundColor(Color.parseColor("#E3F2FD"))
// light blue
                    tvAmount.setTextColor(Color.BLACK)
                    tvDate.setTextColor(Color.BLACK)
                }
            }
        }
    }

    private fun updateData(tvAmount: TextView, tvDate: TextView) {
        val amountValue = when (dataIndex % 3) {
            0 -> 1250.5
            1 -> 98765.75
            else -> 499.0
        }

        // Different data formats: Currency in Indian locale + Date formats
        val currency = NumberFormat.getCurrencyInstance(Locale("en",
"IN")).format(amountValue)

        val dateText = when (dataIndex % 3) {
            0 -> SimpleDateFormat("dd/MM/yyyy",
Locale.getDefault()).format(Date())
            1 -> SimpleDateFormat("MMM dd, yyyy",
Locale.getDefault()).format(Date())
            else -> SimpleDateFormat("EEEE, dd MMM yyyy",
Locale.getDefault()).format(Date())
        }

        tvAmount.text = "Amount: $currency"
        tvDate.text = "Date: $dateText"
    }
}
```

Output



Result

The mobile application UI successfully adapts to different screen sizes and orientations using ConstraintLayout. The design maintains visual hierarchy, responsive behavior, and aesthetic consistency across devices.

Ex. No. : 2 **Date: 28.01.26**

Register No.: 231701032 **Name: Mohamed Zaidh A**

Custom Calculator Design

Aim

Design a scientific calculator with custom UI components. The app should handle user input and offer visual feedback for button presses. Design Focus: Interaction design, visual feedback, aesthetics.

Procedure

1. **Requirement Analysis**
   Identify the objective of developing a **scientific calculator application** with custom UI components that supports numeric input, arithmetic operations, and scientific functions such as sin, cos, tan, log, and square root. Ensure proper interaction design and visual feedback for button presses.
2. **Create an Android Studio Project**
   - Open **Android Studio**.
   - Select **New Project → Empty Activity**.
   - Enter the application name as **Custom Scientific Calculator Design**.
   - Choose **Kotlin** as the programming language.
   - Select **Minimum SDK (API 21 or above)**.
3. **Design the User Interface**
   - Open the layout file `activity_main.xml`.
   - Use **ConstraintLayout or LinearLayout** as the root layout.
   - Create a **TextView** at the top to display the calculator input and results.
   - Add calculator buttons using **GridLayout or LinearLayout rows**.
4. **Add Calculator Buttons**
   - Insert buttons for numeric values **0 – 9**.
   - Add arithmetic operator buttons **+, −, ×, ÷**.
   - Include additional buttons such as **Decimal (.)**, **Clear (C)**, and **Equal (=)**.

- Add scientific function buttons like **sin, cos, tan, log, and** √.

5. **Apply Custom UI Styling**
   - Define custom button styles in **styles.xml**.
   - Apply different colors for:
     - Numeric buttons
     - Operator buttons
     - Scientific buttons
   - Use **background tint, rounded corners, margins, and padding** to enhance the visual appearance.
6. **Implement Button Click Handling**
   - Open the `MainActivity.kt` file.
   - Use **View Binding or findViewById()** to connect UI components.
   - Implement click listeners for numeric buttons to append numbers to the display.
   - Implement click listeners for operators to perform arithmetic calculations.
7. **Implement Scientific Functions**
   - Write functions to perform **sin(), cos(), tan(), log(), and sqrt()** calculations.
   - Use **Kotlin Math functions** to compute the results.
   - Display the result in the calculator display area.
8. **Provide Visual Feedback for Button Presses**
   - Add animation effects such as **button scaling or ripple effects** when buttons are pressed.
   - Change button color or opacity during touch events to improve user interaction.
9. **Handle Errors and Input Validation**
   - Prevent invalid operations such as **division by zero**.
   - Display **Error** messages for invalid calculations.
   - Reset the calculator using the **Clear button**.
10. **Test the Application**

- Run the application in the **Android Emulator** or on a physical device.
- Verify all numeric, arithmetic, and scientific functions.

- Ensure buttons respond correctly with visual feedback.

11. **Build and Run the Application**

- Clean and rebuild the project.
- Execute the application and verify that the custom calculator interface works correctly.

AndroidManifest.xml

```
<?xml version="1.0" encoding="utf-8"?>
<manifest xmlns:android="http://schemas.android.com/apk/res/android"
    xmlns:tools="http://schemas.android.com/tools">

    <application
        android:allowBackup="true"
        android:dataExtractionRules="@xml/data_extraction_rules"
        android:fullBackupContent="@xml/backup_rules"
        android:icon="@mipmap/ic_launcher"
        android:label="@string/app_name"
        android:roundIcon="@mipmap/ic_launcher_round"
        android:supportsRtl="true"
        android:theme="@style/Theme.CustomScientificCalculatorDesign">
        <activity
            android:name=".MainActivity"
            android:exported="true"
            android:label="@string/app_name"
            android:theme="@style/Theme.CustomScientificCalculatorDesign">
            <intent-filter>
                <action android:name="android.intent.action.MAIN" />

                <category android:name="android.intent.category.LAUNCHER" />
            </intent-filter>
        </activity>
    </application>

</manifest>
```

activity_main.xml

```
<?xml version="1.0" encoding="utf-8"?>
<LinearLayout xmlns:android="http://schemas.android.com/apk/res/android"
    xmlns:tools="http://schemas.android.com/tools"
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    android:orientation="vertical"
    android:background="@color/black"
    android:padding="8dp"
    tools:context=".MainActivity">

    <TextView
        android:id="@+id/tvDisplay"
        android:layout_width="match_parent"
```

```
        android:layout_height="0dp"
        android:layout_weight="2"
        android:background="@color/gray_dark"
        android:gravity="bottom|end"
        android:padding="16dp"
        android:text="0"
        android:textColor="@color/white"
        android:textSize="42sp"
        android:textStyle="bold" />

    <LinearLayout
        android:layout_width="match_parent"
        android:layout_height="0dp"
        android:layout_weight="5"
        android:orientation="vertical"
        android:layout_marginTop="8dp">

        <!-- Row 1 -->
        <LinearLayout
            android:layout_width="match_parent"
            android:layout_height="0dp"
            android:layout_weight="1"
            android:orientation="horizontal">

            <Button
                android:id="@+id/btnClear"
                style="@style/CalcButton.Operator"
                android:layout_width="0dp"
                android:layout_height="match_parent"
                android:layout_weight="1"
                android:text="C" />

            <Button
                android:id="@+id/btnSin"
                style="@style/CalcButton.Scientific"
                android:layout_width="0dp"
                android:layout_height="match_parent"
                android:layout_weight="1"
                android:text="sin" />

            <Button
                android:id="@+id/btnCos"
                style="@style/CalcButton.Scientific"
                android:layout_width="0dp"
                android:layout_height="match_parent"
                android:layout_weight="1"
                android:text="cos" />

            <Button
                android:id="@+id/btnTan"
                style="@style/CalcButton.Scientific"
                android:layout_width="0dp"
                android:layout_height="match_parent"
                android:layout_weight="1"
                android:text="tan" />
        </LinearLayout>
```

```
<!-- Row 2 -->
<LinearLayout
    android:layout_width="match_parent"
    android:layout_height="0dp"
    android:layout_weight="1"
    android:orientation="horizontal">

    <Button
        android:id="@+id/btnLog"
        style="@style/CalcButton.Scientific"
        android:layout_width="0dp"
        android:layout_height="match_parent"
        android:layout_weight="1"
        android:text="log" />

    <Button
        android:id="@+id/btnSqrt"
        style="@style/CalcButton.Scientific"
        android:layout_width="0dp"
        android:layout_height="match_parent"
        android:layout_weight="1"
        android:text="√" />

    <Button
        android:id="@+id/btnDiv"
        style="@style/CalcButton.Operator"
        android:layout_width="0dp"
        android:layout_height="match_parent"
        android:layout_weight="1"
        android:text="÷" />

    <Button
        android:id="@+id/btnMul"
        style="@style/CalcButton.Operator"
        android:layout_width="0dp"
        android:layout_height="match_parent"
        android:layout_weight="1"
        android:text="×" />
</LinearLayout>

<!-- Row 3 -->
<LinearLayout
    android:layout_width="match_parent"
    android:layout_height="0dp"
    android:layout_weight="1"
    android:orientation="horizontal">

    <Button
        android:id="@+id/btn7"
        style="@style/CalcButton.Numeric"
        android:layout_width="0dp"
        android:layout_height="match_parent"
        android:layout_weight="1"
        android:text="7" />
```

```
        <Button
            android:id="@+id/btn8"
            style="@style/CalcButton.Numeric"
            android:layout_width="0dp"
            android:layout_height="match_parent"
            android:layout_weight="1"
            android:text="8" />

        <Button
            android:id="@+id/btn9"
            style="@style/CalcButton.Numeric"
            android:layout_width="0dp"
            android:layout_height="match_parent"
            android:layout_weight="1"
            android:text="9" />

        <Button
            android:id="@+id/btnSub"
            style="@style/CalcButton.Operator"
            android:layout_width="0dp"
            android:layout_height="match_parent"
            android:layout_weight="1"
            android:text="-" />
    </LinearLayout>

    <!-- Row 4 -->
    <LinearLayout
        android:layout_width="match_parent"
        android:layout_height="0dp"
        android:layout_weight="1"
        android:orientation="horizontal">

        <Button
            android:id="@+id/btn4"
            style="@style/CalcButton.Numeric"
            android:layout_width="0dp"
            android:layout_height="match_parent"
            android:layout_weight="1"
            android:text="4" />

        <Button
            android:id="@+id/btn5"
            style="@style/CalcButton.Numeric"
            android:layout_width="0dp"
            android:layout_height="match_parent"
            android:layout_weight="1"
            android:text="5" />

        <Button
            android:id="@+id/btn6"
            style="@style/CalcButton.Numeric"
            android:layout_width="0dp"
            android:layout_height="match_parent"
            android:layout_weight="1"
            android:text="6" />
```

```
        <Button
            android:id="@+id/btnAdd"
            style="@style/CalcButton.Operator"
            android:layout_width="0dp"
            android:layout_height="match_parent"
            android:layout_weight="1"
            android:text="+" />
    </LinearLayout>

    <!-- Row 5 -->
    <LinearLayout
        android:layout_width="match_parent"
        android:layout_height="0dp"
        android:layout_weight="1"
        android:orientation="horizontal">

        <Button
            android:id="@+id/btn1"
            style="@style/CalcButton.Numeric"
            android:layout_width="0dp"
            android:layout_height="match_parent"
            android:layout_weight="1"
            android:text="1" />

        <Button
            android:id="@+id/btn2"
            style="@style/CalcButton.Numeric"
            android:layout_width="0dp"
            android:layout_height="match_parent"
            android:layout_weight="1"
            android:text="2" />

        <Button
            android:id="@+id/btn3"
            style="@style/CalcButton.Numeric"
            android:layout_width="0dp"
            android:layout_height="match_parent"
            android:layout_weight="1"
            android:text="3" />

        <Button
            android:id="@+id/btnEqual"
            style="@style/CalcButton.Operator"
            android:layout_width="0dp"
            android:layout_height="match_parent"
            android:layout_weight="1"
            android:text="=" />
    </LinearLayout>

    <!-- Row 6 -->
    <LinearLayout
        android:layout_width="match_parent"
        android:layout_height="0dp"
        android:layout_weight="1"
        android:orientation="horizontal">
```

```
            <Button
                android:id="@+id/btn0"
                style="@style/CalcButton.Numeric"
                android:layout_width="0dp"
                android:layout_height="match_parent"
                android:layout_weight="2"
                android:text="0" />

            <Button
                android:id="@+id/btnDot"
                style="@style/CalcButton.Numeric"
                android:layout_width="0dp"
                android:layout_height="match_parent"
                android:layout_weight="1"
                android:text="." />

            <Button
                android:id="@+id/btnPercent"
                style="@style/CalcButton.Operator"
                android:layout_width="0dp"
                android:layout_height="match_parent"
                android:layout_weight="1"
                android:text="%" />
        </LinearLayout>

    </LinearLayout>

</LinearLayout>
```

MainActivity.kt

```
package com.example.customscientificcalculatordesign

import android.os.Bundle
import android.view.MotionEvent
import android.widget.Button
import androidx.appcompat.app.AppCompatActivity
import
com.example.customscientificcalculatordesign.databinding.ActivityMainBinding
import java.text.DecimalFormat
import kotlin.math.*

class MainActivity : AppCompatActivity() {

    private lateinit var binding: ActivityMainBinding
    private var lastNumeric: Boolean = false
    private var stateError: Boolean = false
    private var lastDot: Boolean = false

    private val decimalFormat = DecimalFormat("#.#######")

    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)
        binding = ActivityMainBinding.inflate(layoutInflater)
        setContentView(binding.root)

        setNumericOnClickListeners()
        setOperatorOnClickListeners()
        setScientificOnClickListeners()

        val allButtons = listOf(
            binding.btn0, binding.btn1, binding.btn2, binding.btn3,
            binding.btn4, binding.btn5, binding.btn6, binding.btn7,
            binding.btn8, binding.btn9, binding.btnAdd, binding.btnSub,
            binding.btnMul, binding.btnDiv, binding.btnDot, binding.btnEqual,
            binding.btnClear, binding.btnSin, binding.btnCos, binding.btnTan,
            binding.btnLog, binding.btnSqrt
        )

        for (button in allButtons) {
            applyPressEffect(button)
        }

        binding.btnClear.setOnClickListener {
            binding.tvDisplay.text = "0"
            lastNumeric = false
            stateError = false
            lastDot = false
        }

        binding.btnDot.setOnClickListener {
            if (lastNumeric && !stateError && !lastDot) {
                binding.tvDisplay.append(".")
                lastNumeric = false
                lastDot = true
```

```
            } else if (!lastNumeric && !stateError && !lastDot) {
                binding.tvDisplay.append("0.")
                lastNumeric = false
                lastDot = true
            }
        }

        binding.btnEqual.setOnClickListener {
            onEqual()
        }
    }

    private fun applyPressEffect(button: Button) {
        button.setOnTouchListener { v, event ->
            when (event.action) {
                MotionEvent.ACTION_DOWN -> {

v.animate().scaleX(0.92f).scaleY(0.92f).setDuration(80).start()
                    v.alpha = 0.8f
                }
                MotionEvent.ACTION_UP, MotionEvent.ACTION_CANCEL -> {
                    v.animate().scaleX(1f).scaleY(1f).setDuration(80).start()
                    v.alpha = 1f
                }
            }
            false
        }
    }

    private fun setNumericOnClickListeners() {
        val buttons = listOf(
            binding.btn0, binding.btn1, binding.btn2, binding.btn3,
            binding.btn4, binding.btn5, binding.btn6, binding.btn7,
            binding.btn8, binding.btn9
        )
        for (button in buttons) {
            button.setOnClickListener {
                if (stateError) {
                    binding.tvDisplay.text = (it as Button).text
                     stateError = false
                } else {
                    val currentText = binding.tvDisplay.text.toString()
                    if (currentText == "0") {
                        binding.tvDisplay.text = (it as Button).text
                     } else {
                        binding.tvDisplay.append((it as Button).text)
                    }
                }
                lastNumeric = true
            }
        }
    }

    private fun setOperatorOnClickListeners() {
        val buttons = listOf(
            binding.btnAdd, binding.btnSub, binding.btnMul, binding.btnDiv
```

```
        )
        for (button in buttons) {
            button.setOnClickListener {
                if (lastNumeric && !stateError) {
                    binding.tvDisplay.append(" " + (it as Button).text + " ")
                    lastNumeric = false
                    lastDot = false
                }
            }
        }
    }

    private fun setScientificOnClickListeners() {
        binding.btnSin.setOnClickListener { calculateScientific {
sin(Math.toRadians(it)) } }
        binding.btnCos.setOnClickListener { calculateScientific {
cos(Math.toRadians(it)) } }
        binding.btnTan.setOnClickListener {
            calculateScientific {
                if (it % 180 == 90.0 || it % 180 == -90.0) Double.NaN
                else tan(Math.toRadians(it))
            }
        }
        binding.btnLog.setOnClickListener { calculateScientific { if (it > 0)
log10(it) else Double.NaN } }
        binding.btnSqrt.setOnClickListener { calculateScientific { if (it >= 0)
sqrt(it) else Double.NaN } }
    }

    private fun calculateScientific(operation: (Double) -> Double) {
        if (!stateError) {
            try {
                val currentText = binding.tvDisplay.text.toString()
                if (!lastNumeric) {
                    binding.tvDisplay.text = "Error"
                    stateError = true
                    lastNumeric = false
                    return
                }

                val valueToProcess = evaluate(currentText)
                val result = operation(valueToProcess)

                if (result.isNaN() || result.isInfinite()) {
                    binding.tvDisplay.text = "Error"
                    stateError = true
                    lastNumeric = false
                } else {
                    binding.tvDisplay.text = decimalFormat.format(result)
                    lastNumeric = true
                    stateError = false
                    lastDot = binding.tvDisplay.text.contains(".")
                }
            } catch (e: Exception) {
                binding.tvDisplay.text = "Error"
                stateError = true
```

```
            lastNumeric = false
        }
    }
}

private fun onEqual() {
    if (lastNumeric && !stateError) {
        val text = binding.tvDisplay.text.toString()
        try {
            val result = evaluate(text)
            if (result.isInfinite() || result.isNaN()) {
                binding.tvDisplay.text = "Error"
                stateError = true
                lastNumeric = false
            } else {
                binding.tvDisplay.text = decimalFormat.format(result)
                lastDot = binding.tvDisplay.text.contains(".")
                lastNumeric = true
                stateError = false
            }
        } catch (e: Exception) {
            binding.tvDisplay.text = "Error"
            stateError = true
            lastNumeric = false
        }
    }
}

private fun evaluate(expression: String): Double {
    val tokens = expression.trim().split("\\s+".toRegex())
    if (tokens.isEmpty()) return 0.0
    if (tokens.size == 1) return tokens[0].toDouble()

    val values = mutableListOf<Double>()
    val ops = mutableListOf<String>()

    fun precedence(op: String): Int {
        return when (op) {
            "+", "−" -> 1
            "×", "÷" -> 2
            else -> 0
        }
    }

    fun applyOperation() {
        if (values.size < 2 || ops.isEmpty()) return
        val b = values.removeAt(values.lastIndex)
        val a = values.removeAt(values.lastIndex)
        val op = ops.removeAt(ops.lastIndex)

        val result = when (op) {
            "+" -> a + b
            "−" -> a - b
            "×" -> a * b
            "÷" -> if (b == 0.0) Double.NaN else a / b
            else -> 0.0
```

```
            }
            values.add(result)
        }

        for (token in tokens) {
            if (token.toDoubleOrNull() != null) {
                values.add(token.toDouble())
            } else {
                while (ops.isNotEmpty() && precedence(ops.last()) >=
precedence(token)) {
                    applyOperation()
                }
                ops.add(token)
            }
        }

        while (ops.isNotEmpty()) {
            applyOperation()
        }

        return values.firstOrNull() ?: 0.0
    }
}
```

Colors.xml

```
<?xml version="1.0" encoding="utf-8"?>
<resources>
    <color name="black">#FF000000</color>
    <color name="white">#FFFFFFFF</color>
    <color name="gray_dark">#212121</color>
    <color name="gray_medium">#424242</color>
    <color name="gray_light">#616161</color>
    <color name="accent_blue">#2196F3</color>
    <color name="accent_orange">#FF9800</color>
</resources>
```

Themes.xml

```
<?xml version="1.0" encoding="utf-8"?>
<resources xmlns:tools="http://schemas.android.com/tools">

    <style name="Theme.CustomScientificCalculatorDesign"
parent="Theme.MaterialComponents.DayNight.NoActionBar">
        <item name="colorPrimary">@color/gray_dark</item>
        <item name="colorPrimaryVariant">@color/black</item>
        <item name="colorOnPrimary">@color/white</item>
        <item name="colorSecondary">@color/accent_blue</item>
        <item name="colorSecondaryVariant">@color/accent_blue</item>
        <item name="colorOnSecondary">@color/white</item>
        <item name="android:statusBarColor">@color/black</item>
        <item name="android:navigationBarColor">@color/black</item>
    </style>

    <style name="CalcButton" parent="Widget.MaterialComponents.Button">
```

```
        <item name="android:layout_width">0dp</item>
        <item name="android:layout_height">0dp</item>
        <item name="android:layout_margin">4dp</item>
        <item name="android:layout_gravity">fill</item>
        <item name="android:minHeight">64dp</item>
        <item name="android:textSize">18sp</item>
        <item name="android:textStyle">bold</item>
        <item name="android:textColor">@color/white</item>
        <item name="android:insetTop">0dp</item>
        <item name="android:insetBottom">0dp</item>
        <item name="cornerRadius">16dp</item>
        <item name="backgroundTint">@color/gray_medium</item>
        <item name="rippleColor">@color/accent_blue</item>
    </style>

    <style name="CalcButton.Numeric" parent="CalcButton">
        <item name="backgroundTint">@color/accent_orange</item>
        <item name="android:textColor">@color/white</item>
    </style>

    <style name="CalcButton.Operator" parent="CalcButton">
        <item name="backgroundTint">@color/accent_orange</item>
        <item name="android:textColor">@color/white</item>
    </style>

    <style name="CalcButton.Scientific" parent="CalcButton">
        <item name="backgroundTint">@color/gray_dark</item>
        <item name="android:textColor">@color/white</item>
    </style>

    <style name="CalcButton.Equal" parent="CalcButton">
        <item name="backgroundTint">@color/accent_blue</item>
        <item name="android:textColor">@color/white</item>
    </style>

</resources>
```

Output

Result

Thus, a **custom scientific calculator application** was successfully designed and developed with user input handling, scientific functions, and visual feedback for button presses.

Ex. No. : 3 **Date: 18.02.26**

Register No.: 231701032 **Name: Mohamed Zaidh A**

Wire framing and Prototyping with Figma

Aim

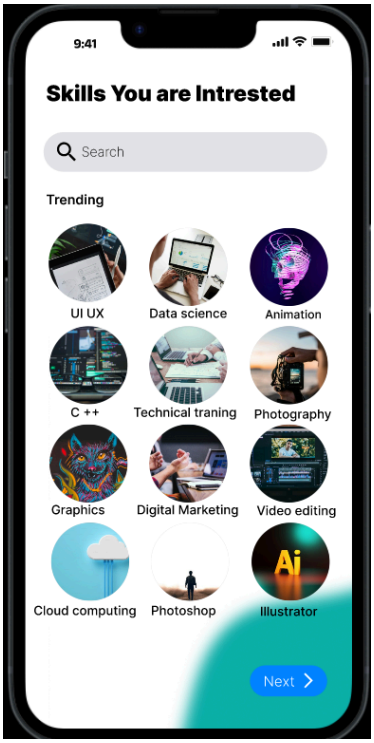
Create wireframes and a functional prototype for a mobile app using Figma. Convert the prototype into a basic Android application. Design Focus: Prototyping, flow consistency.

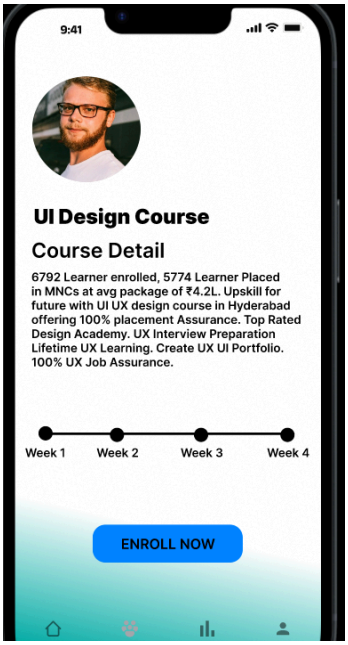
1. **Requirement Analysis**
    a. Identified the essential features required for the Learning Hub app:
        a. Course category selection
        b. Course details page
        c. Video lessons and study materials
        d. Progress tracking dashboard
        e. Quiz and assessment section
        f. Profile and back navigation
2. **Creating Frames in Figma**
3. Opened Figma and selected the **iPhone 14 & 15 Pro Max** frame size.
    a. Designed multiple key screens:
        a. Home / Course Categories Screen
        b. Course Details Screen
        c. Learning Module (Video/Content) Screen
        d. Quiz Screen
        e. Progress Dashboard Screen
4. **Wireframing**
5. Created low-fidelity wireframes using simple shapes like rectangles and placeholders for images and text.
6. Organized content sections clearly, including course cards, navigation bar, and action buttons.
7. Ensured proper spacing, alignment, and touch-friendly button sizes for better usability.
8. **High-Fidelity Design**
9. Applied a consistent color palette and modern UI theme suitable for an educational platform.
10. Added rounded cards, icons, illustrations, and clear typography for better readability.

11. Established strong visual hierarchy by highlighting important elements like “Start Learning,” “Continue,” and “Submit Quiz.”
12. Maintained uniform spacing, consistent fonts, and cohesive design components across all screens.
13. **Prototyping**
14. Used the Prototype tab in Figma to add interactivity.
    a. Configured interactions such as:
        a. Course card → Navigate to Course Details Screen
        b. Start Learning button → Navigate to Learning Module Screen
        c. Take Quiz button → Navigate to Quiz Screen
        d. Back button → Return to previous screen
15. Added smooth transition animations and tested the flow using Present mode.
16. **Testing and Refinement**
17. Reviewed layout alignment and overall design consistency.
18. Improved text readability, color contrast, and accessibility.
19. Verified seamless navigation between screens to ensure a smooth user experience.

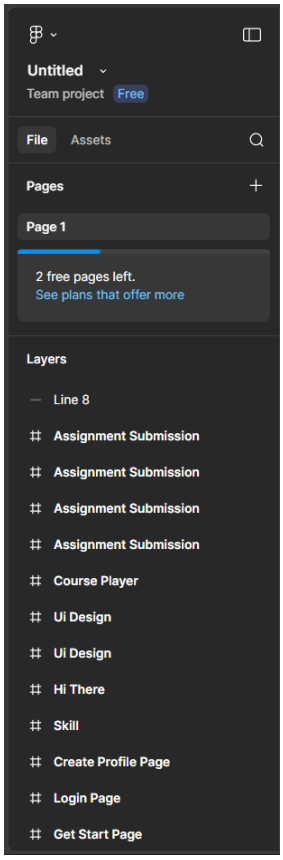
Output

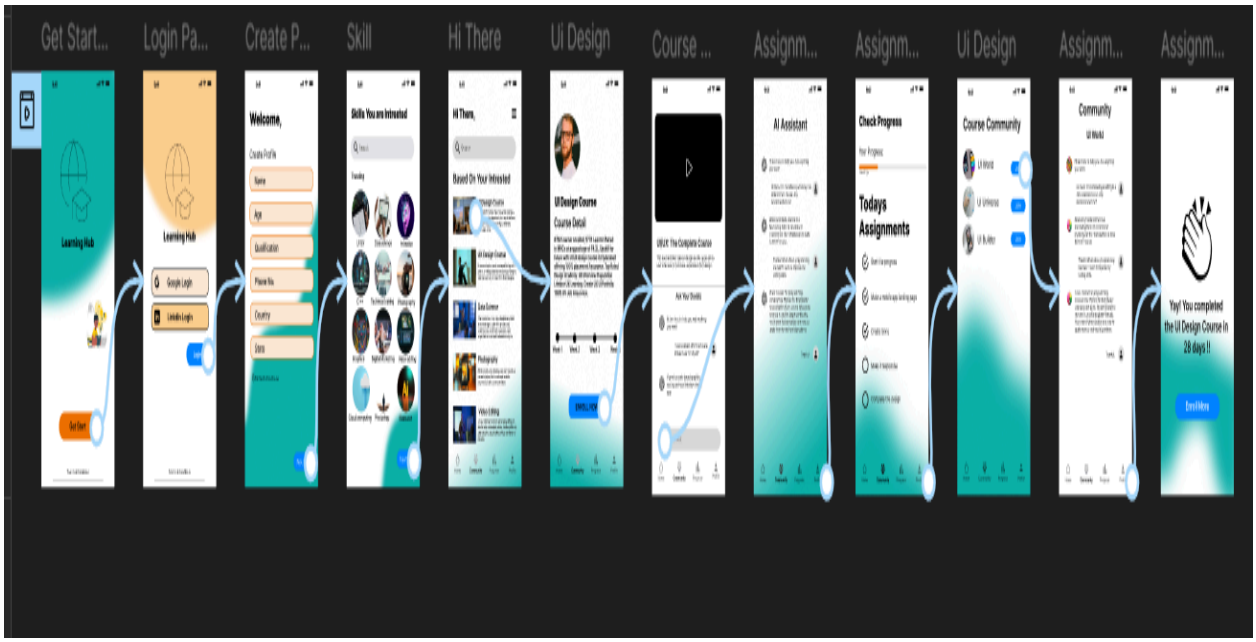

9:41
Learning Hub
Get Start
Tearms & Conditions


9:41
Skills You are Intrested
Search
Trending
UI UX
Data science
Animation
C ++
Technical traning
Photography
Graphics
Digital Marketing
Video editing
Cloud computing
Photoshop
Illustrator
Next


9:41
UI Design Course
Course Detail
6792 Learner enrolled, 5774 Learner Placed in MNCs at avg package of ₹4.2L. Upskill for future with UI UX design course in Hyderabad offering 100% placement Assurance. Top Rated Design Academy. UX Interview Preparation Lifetime UX Learning. Create UX UI Portfolio. 100% UX Job Assurance.
Week 1
Week 2
Week 3
Week 4
ENROLL NOW


9:41
UI/UX: The Complete Course
This specialization takes a design-centric approach to user interface (UI) and user experience (UX) design.
Ask Your Doubts
Hi Iam here to help you. Ask anything you want!
I have a doubt. Which software should I use for UI/UX?
Figma is a web-based graphics editing and user interface design app.
Type to chat
Home
Community
Progress
Profile


Untitled
Team project
Free
File
Assets
Pages
Page 1
2 free pages left.
See plans that offer more
Layers
Line 8
Assignment Submission
Assignment Submission
Assignment Submission
Assignment Submission
Course Player
Ui Design
Ui Design
Hi There
Skill
Create Profile Page
Login Page
Get Start Page


Get Start...
Login Pa...
Create P...
Skill
Hi There
Ui Design
Course ...
Assignm...
Assignm...
Ui Design
Assignm...
Assignm...
Learning Hub
Welcome,
Skills You are Intrested
Hi There,
Based On Your Intrested
UI Design Course
Course Detail
AI Assistant
Check Progress
Todays Assignments
Course Community
Community
Yay! You completed the UI Design Course in 28 days !!

Result:

The TV Remote Mobile Application was successfully wireframed and prototyped using Figma. The application includes interactive screens for brand selection, remote control functions, and numeric input. The prototype demonstrates effective UI design principles, smooth navigation flow, and a user-friendly interface.

Ex. No. : 4 **Date: 25.02.26**

Register No.: 231701032 **Name: Mohamed Zaidh A**

Gesture-Based Interaction App

Aim

Implement gesture-based interactions such as swiping, pinch-to-zoom, and double-tap to navigate different screens or trigger actions in the app. Design Focus: Gesture interaction, smooth transitions.

Procedure

1. **Requirement Analysis**
   Identify the objective of developing a mobile application that demonstrates **gesture-based interactions** such as swipe, double-tap, and pinch-to-zoom. These gestures will be used to trigger actions such as navigating between screens, zooming UI elements, and displaying feedback messages.
2. **Create an Android Studio Project**
   - Open **Android Studio**.
   - Select **New Project → Empty Activity**.
   - Enter the application name as **Gesture-Based Interaction App**.
   - Choose **Kotlin** as the programming language.
   - Select **Minimum SDK (API 21 or above)**.
3. **Design the User Interface for Main Screen**
   - Open the layout file `activity_main.xml`.
   - Use **ConstraintLayout** as the root layout.
   - Add a **TextView** to display gesture messages and instructions.
   - Add an **ImageView** that will respond to pinch-to-zoom and double-tap gestures.
   - Align UI components properly using constraints.
4. **Design the Second Screen Layout**
   - Create another layout file `activity_second.xml`.
   - Add a **TextView** to indicate that the user has navigated to the second screen.

- Position the TextView at the center using **ConstraintLayout** constraints.

5. **Implement Gesture Detection in MainActivity**
   - Open the `MainActivity.kt` file.
   - Create objects of **GestureDetector** to detect gestures like swipe and double-tap.
   - Create an object of **ScaleGestureDetector** to detect pinch-to-zoom gestures.
6. **Implement Double Tap Gesture**
   - Override the **onDoubleTap()** method of `GestureDetector.SimpleOnGestureListener`.
   - Trigger navigation to the **SecondActivity** using an **Intent** when the user performs a double tap.
7. **Implement Swipe Gesture**
   - Override the **onFling()** method of the gesture listener.
   - Detect swipe directions such as **swipe left, swipe right, swipe up, and swipe down**.
   - Perform actions such as navigating to the second screen or resetting UI elements based on swipe direction.
8. **Implement Pinch-to-Zoom Gesture**
   - Use **ScaleGestureDetector.SimpleOnScaleGestureListener**.
   - Override the **onScale()** method to detect pinch gestures.
   - Adjust the scale of the **ImageView** or change the text size dynamically during zoom gestures.
9. **Handle Touch Events**
   - Override the **setOnTouchListener()** method for the TextView and ImageView.
   - Pass touch events to the **GestureDetector** and **ScaleGestureDetector** to process gestures correctly.
10. **Provide Visual Feedback**

- Update the TextView text or display **Toast messages** when gestures are detected.
- Apply smooth transitions between screens using animation methods such as `overridePendingTransition()`.

11. **Test the Application**

- Run the application in the **Android Emulator or on a physical device**.
- Test the following gestures:

- Swipe gestures for navigation or actions.
- Double-tap gesture to open the second screen.
- Pinch-to-zoom gesture to zoom text or images.

12. **Build and Run the Application**

- Clean and rebuild the project.
- Execute the application and verify that all gesture-based interactions work smoothly.

AndroidManifest.xml

```
<?xml version="1.0" encoding="utf-8"?>
<manifest xmlns:android="http://schemas.android.com/apk/res/android"
    xmlns:tools="http://schemas.android.com/tools">

    <application
        android:allowBackup="true"
        android:dataExtractionRules="@xml/data_extraction_rules"
        android:fullBackupContent="@xml/backup_rules"
        android:icon="@mipmap/ic_launcher"
        android:label="@string/app_name"
        android:roundIcon="@mipmap/ic_launcher_round"
        android:supportsRtl="true"
        android:theme="@style/Theme.Ex4Demo"
        >
        <activity
            android:name=".SecondActivity"
            android:exported="false" />
        <activity
            android:name=".MainActivity"
            android:exported="true">
            <intent-filter>
                <action android:name="android.intent.action.MAIN" />

                <category android:name="android.intent.category.LAUNCHER" />
            </intent-filter>
        </activity>
    </application>

</manifest>
```

activity_main.xml

```
<?xml version="1.0" encoding="utf-8"?>
<androidx.constraintlayout.widget.ConstraintLayout
xmlns:android="http://schemas.android.com/apk/res/android"
    xmlns:app="http://schemas.android.com/apk/res-auto"
    xmlns:tools="http://schemas.android.com/tools"
    android:id="@+id/main"
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    android:padding="16dp"
    tools:context=".MainActivity">
```

```
    <TextView
        android:id="@+id/tv"
        android:layout_width="0dp"
        android:layout_height="wrap_content"
        android:text="Use swipe, double tap, and pinch gestures"
        android:textAlignment="center"
        android:textSize="24sp"
        android:textStyle="bold"
        app:layout_constraintTop_toTopOf="parent"
        app:layout_constraintBottom_toTopOf="@+id/gesture_image"
        app:layout_constraintStart_toStartOf="parent"
        app:layout_constraintEnd_toEndOf="parent"
        app:layout_constraintVertical_bias="0.3" />

    <ImageView
        android:id="@+id/gesture_image"
        android:layout_width="220dp"
        android:layout_height="220dp"
        android:layout_marginTop="24dp"
        android:contentDescription="Gesture interaction image"
        android:src="@mipmap/image_0"
        app:layout_constraintTop_toBottomOf="@id/tv"
        app:layout_constraintStart_toStartOf="parent"
        app:layout_constraintEnd_toEndOf="parent" />

    <TextView
        android:id="@+id/tvHint"
        android:layout_width="0dp"
        android:layout_height="wrap_content"
        android:layout_marginTop="24dp"
        android:text="Swipe left: Open second screen | Swipe right: Reset |
Double tap: Open/Zoom | Pinch: Zoom"
        android:textAlignment="center"
        android:textSize="16sp"
        app:layout_constraintTop_toBottomOf="@id/gesture_image"
        app:layout_constraintStart_toStartOf="parent"
        app:layout_constraintEnd_toEndOf="parent" />

</androidx.constraintlayout.widget.ConstraintLayout>
```

activity_second.xml

```
<?xml version="1.0" encoding="utf-8"?>
<androidx.constraintlayout.widget.ConstraintLayout
xmlns:android="http://schemas.android.com/apk/res/android"
    xmlns:app="http://schemas.android.com/apk/res-auto"
    xmlns:tools="http://schemas.android.com/tools"
    android:id="@+id/secondRoot"
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    android:padding="16dp"
    tools:context=".SecondActivity">

    <TextView
        android:id="@+id/tv2"
        android:layout_width="0dp"
```

```
        android:layout_height="wrap_content"
        android:text="Second Screen\nSwipe right to go back"
        android:textAlignment="center"
        android:textSize="26sp"
        android:textStyle="bold"
        app:layout_constraintTop_toTopOf="parent"
        app:layout_constraintBottom_toBottomOf="parent"
        app:layout_constraintStart_toStartOf="parent"
        app:layout_constraintEnd_toEndOf="parent" />

</androidx.constraintlayout.widget.ConstraintLayout>
```

MainActivity.kt

```
package com.example.ex4demo

import android.annotation.SuppressLint
import android.content.Intent
import android.os.Bundle
import android.util.TypedValue
import android.view.GestureDetector
import android.view.MotionEvent
import android.view.ScaleGestureDetector
import android.widget.ImageView
import android.widget.TextView
import android.widget.Toast
import androidx.activity.enableEdgeToEdge
import androidx.appcompat.app.AppCompatActivity
import androidx.constraintlayout.widget.ConstraintLayout
import androidx.core.view.ViewCompat
import androidx.core.view.WindowInsetsCompat

class MainActivity : AppCompatActivity() {

    private lateinit var textGestureDetector: GestureDetector
    private lateinit var textScaleDetector: ScaleGestureDetector

    private lateinit var imageGestureDetector: GestureDetector
    private lateinit var imageScaleDetector: ScaleGestureDetector

    private lateinit var swipeGestureDetector: GestureDetector

    private var currentTextSizeSp = 24f
    private var imageScale = 1.0f
    private val minScale = 0.5f
    private val maxScale = 4.0f

    @SuppressLint("ClickableViewAccessibility")
    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)
        enableEdgeToEdge()
        setContentView(R.layout.activity_main)

        ViewCompat.setOnApplyWindowInsetsListener(findViewById(R.id.main)) { v,
insets ->
            val systemBars =
```

```
insets.getInsets(WindowInsetsCompat.Type.systemBars())
            v.setPadding(systemBars.left, systemBars.top, systemBars.right,
systemBars.bottom)
            insets
        }

        val root = findViewById<ConstraintLayout>(R.id.main)
        val tv = findViewById<TextView>(R.id.tv)
        val tvHint = findViewById<TextView>(R.id.tvHint)
        val img = findViewById<ImageView>(R.id.gesture_image)

        // Swipe detector for whole screen
        swipeGestureDetector = GestureDetector(this,
            object : GestureDetector.SimpleOnGestureListener() {
                private val swipeThreshold = 100
                private val swipeVelocityThreshold = 100

                override fun onFling(
                    e1: MotionEvent?,
                    e2: MotionEvent,
                    velocityX: Float,
                    velocityY: Float
                ): Boolean {
                    if (e1 == null) return false

                    val diffX = e2.x - e1.x
                     val diffY = e2.y - e1.y

                     return if (kotlin.math.abs(diffX) > kotlin.math.abs(diffY))
{
                        if (kotlin.math.abs(diffX) > swipeThreshold &&
                            kotlin.math.abs(velocityX) > swipeVelocityThreshold
                        ) {
                            if (diffX > 0) {
                                tv.text = "Swipe Right Detected"
                                tvHint.text = "Image and text reset"
                                resetUI(tv, img)
                            } else {
                                tv.text = "Swipe Left Detected"
                                tvHint.text = "Opening second screen"
                                Toast.makeText(
                                    this@MainActivity,
                                    "Opening Second Screen",
                                    Toast.LENGTH_SHORT
                                 ).show()
                                startActivity(Intent(this@MainActivity,
SecondActivity::class.java))
                                overridePendingTransition(
                                    android.R.anim.slide_in_left,
                                    android.R.anim.slide_out_right
                                 )
                            }
                            true
                        } else {
                            false
                        }
```

```
                    } else {
                        if (kotlin.math.abs(diffY) > swipeThreshold &&
                            kotlin.math.abs(velocityY) > swipeVelocityThreshold
                        ) {
                            if (diffY > 0) {
                                tv.text = "Swipe Down Detected"
                                tvHint.text = "Image zoom reset"
                                imageScale = 1.0f

img.animate().scaleX(imageScale).scaleY(imageScale).setDuration(200).start()
                            } else {
                                tv.text = "Swipe Up Detected"
                                tvHint.text = "Image enlarged slightly"
                                imageScale = (imageScale +
0.2f).coerceIn(minScale, maxScale)

img.animate().scaleX(imageScale).scaleY(imageScale).setDuration(200).start()
                            }
                            true
                        } else {
                            false
                        }
                    }
                }
            })

        // Text double tap detector
        textGestureDetector = GestureDetector(this,
            object : GestureDetector.SimpleOnGestureListener() {
                override fun onDoubleTap(e: MotionEvent): Boolean {
                    tv.text = "Double Tap Detected"
                    tvHint.text = "Opening second screen"
                    startActivity(Intent(this@MainActivity,
SecondActivity::class.java))
                    overridePendingTransition(android.R.anim.fade_in,
android.R.anim.fade_out)
                    return true
                }
            })

        // Text pinch detector
        textScaleDetector = ScaleGestureDetector(this,
            object : ScaleGestureDetector.SimpleOnScaleGestureListener() {
                override fun onScale(detector: ScaleGestureDetector): Boolean {
                    currentTextSizeSp *= detector.scaleFactor
                     currentTextSizeSp = currentTextSizeSp.coerceIn(16f, 60f)
                    tv.setTextSize(TypedValue.COMPLEX_UNIT_SP,
currentTextSizeSp)
                    tvHint.text = "Pinch detected on text"
                    return true
                }
            })

        tv.setOnTouchListener { _, event ->
            textGestureDetector.onTouchEvent(event)
            textScaleDetector.onTouchEvent(event)
```

```
            swipeGestureDetector.onTouchEvent(event)
            true
        }

        // Image pinch detector
        imageScaleDetector = ScaleGestureDetector(this,
            object : ScaleGestureDetector.SimpleOnScaleGestureListener() {
                override fun onScale(detector: ScaleGestureDetector): Boolean {
                    val newScale = (imageScale *
detector.scaleFactor).coerceIn(minScale, maxScale)

                    img.pivotX = detector.focusX
                    img.pivotY = detector.focusY
                    imageScale = newScale

                    img.animate()
                        .scaleX(imageScale)
                        .scaleY(imageScale)
                        .setDuration(100)
                        .start()

                    tvHint.text = "Pinch detected on image"
                    return true
                }
            })

        // Image double tap detector
        imageGestureDetector = GestureDetector(this,
            object : GestureDetector.SimpleOnGestureListener() {
                override fun onDoubleTap(e: MotionEvent): Boolean {
                    img.pivotX = e.x
                    img.pivotY = e.y

                    imageScale = if (imageScale < 2f) 2.5f else 1.0f

                    img.animate()
                        .scaleX(imageScale)
                        .scaleY(imageScale)
                        .setDuration(250)
                        .start()

                    tvHint.text = "Double tap detected on image"
                    return true
                }
            })

        img.setOnTouchListener { _, event ->
            imageGestureDetector.onTouchEvent(event)
            imageScaleDetector.onTouchEvent(event)
            swipeGestureDetector.onTouchEvent(event)
            true
        }

        root.setOnTouchListener { _, event ->
            swipeGestureDetector.onTouchEvent(event)
            true
```

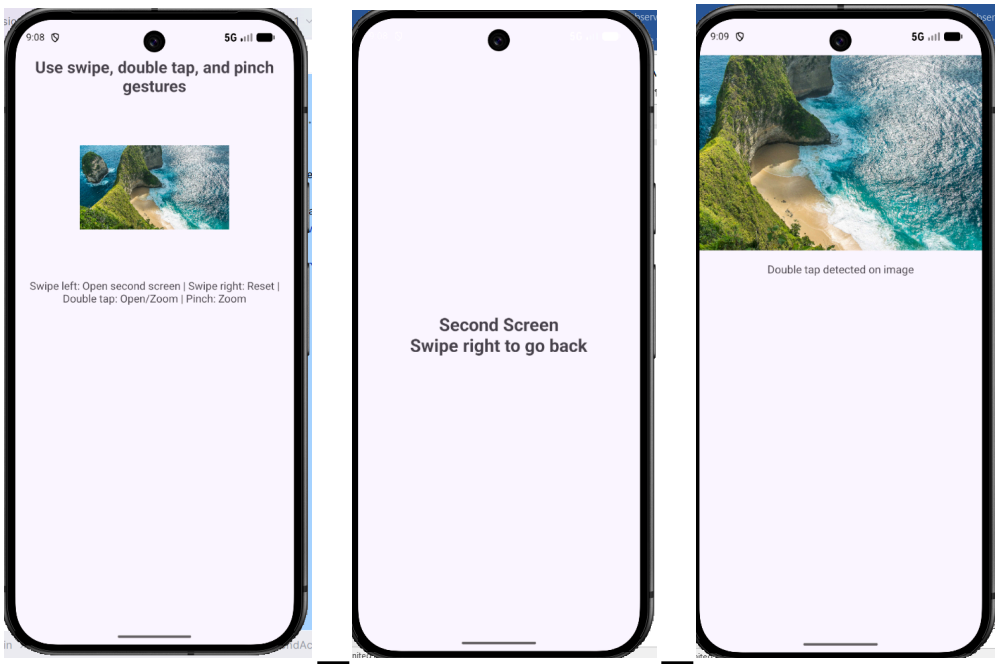
```
        }
    }

    private fun resetUI(tv: TextView, img: ImageView) {
        currentTextSizeSp = 24f
        tv.setTextSize(TypedValue.COMPLEX_UNIT_SP, currentTextSizeSp)
        tv.text = "Use swipe, double tap, and pinch gestures"

        imageScale = 1.0f
        img.animate()
            .scaleX(imageScale)
            .scaleY(imageScale)
            .setDuration(200)
            .start()
    }
}
```

SecondActivity.kt

```
package com.example.ex4demo

import android.os.Bundle
import android.view.GestureDetector
import android.view.MotionEvent
import android.widget.TextView
import android.widget.Toast
import androidx.appcompat.app.AppCompatActivity
import androidx.constraintlayout.widget.ConstraintLayout

class SecondActivity : AppCompatActivity() {

    private lateinit var swipeDetector: GestureDetector

    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)
        setContentView(R.layout.activity_second)

        val root = findViewById<ConstraintLayout>(R.id.secondRoot)
        val tv2 = findViewById<TextView>(R.id.tv2)

        swipeDetector = GestureDetector(this,
            object : GestureDetector.SimpleOnGestureListener() {
                private val swipeThreshold = 100
                private val swipeVelocityThreshold = 100

                override fun onDoubleTap(e: MotionEvent): Boolean {
                    Toast.makeText(
                        this@SecondActivity,
                        "Double Tap on Second Screen",
                        Toast.LENGTH_SHORT
                    ).show()
                    return true
                }

                override fun onFling(
                    e1: MotionEvent?,
```

```
                    e2: MotionEvent,
                    velocityX: Float,
                    velocityY: Float
                ): Boolean {
                    if (e1 == null) return false

                    val diffX = e2.x - e1.x

                    return if (kotlin.math.abs(diffX) > swipeThreshold &&
                        kotlin.math.abs(velocityX) > swipeVelocityThreshold
                    ) {
                        if (diffX > 0) {
                            tv2.text = "Swipe Right Detected\nReturning to Main
Screen"
                            finish()
                            overridePendingTransition(
                                android.R.anim.slide_in_left,
                                android.R.anim.slide_out_right
                            )
                        }
                        true
                    } else {
                        false
                    }
                }
            })

        root.setOnTouchListener { _, event ->
            swipeDetector.onTouchEvent(event)
            true
        }

        tv2.setOnTouchListener { _, event ->
            swipeDetector.onTouchEvent(event)
            true
        }
    }
}
```

Output



Result

Thus, a **gesture-based interaction application** was successfully developed to detect swipe, double-tap, and pinch-to-zoom gestures and trigger corresponding actions with smooth transitions.

Ex. No. : 5 **Date: 04.03.26**

Register No.: 231701032 **Name: Mohamed Zaidh A**

Data Visualization App

Aim

Design and develop a mobile app that displays user data (e.g., steps, expenses) using custom-designed charts and graphs. Design Focus: Data visualization, clarity.

Procedure

1. **Requirement Analysis**
   Identify the objective of developing a mobile application that displays user financial data such as **daily expenses** using multiple charts and graphs. The aim is to represent the data clearly through visual components such as **bar chart, line chart, and pie chart**.
2. **Create an Android Studio Project**
   - Open **Android Studio**.
   - Select **New Project → Empty Activity**.
   - Enter the application name as **Data Visualization App**.
   - Choose **Kotlin** as the programming language.
   - Select **Minimum SDK (API 21 or above)**.
3. **Add Required Library Dependency**
   - Open the **build.gradle (Module: app)** file.
   - Add the dependency for **MPAndroidChart** library.
   - Sync the project to download and configure the chart library.
4. **Design the User Interface**
   - Open the layout file `activity_main.xml`.
   - Use a **ScrollView** as the root layout to support vertical scrolling.
   - Add a **ConstraintLayout** inside the ScrollView.
   - Place a **TextView** for the dashboard title.
   - Add summary TextViews to display:

        - Total Expense
        - Highest Spending Day
    - Add a **Spinner** to select the day of the week.
    - Add a **TextInputEditText** to enter the expense amount.
    - Add a **Button** to update the dashboard.
    - Add **BarChart, LineChart, and PieChart** components to visualize the expense data.
5. **Initialize the Dataset**
    - In the `MainActivity.kt` file, create a list of day labels such as **Mon, Tue, Wed, Thu, Fri, Sat, Sun**.
    - Initialize a mutable list of sample expense values for the seven days.
    - Use these sample values to display charts immediately when the app starts.
6. **Configure the Spinner**
    - Create an **ArrayAdapter** using the list of day labels.
    - Set the adapter to the Spinner so that the user can choose a day for updating the expense value.
7. **Configure the Chart Components**
    - Initialize the **BarChart**, **LineChart**, and **PieChart** in the activity.
    - Disable unnecessary chart descriptions.
    - Configure X-axis labels using the list of days.
    - Disable the right Y-axis where required.
    - Set minimum axis values and enable chart animations for better presentation.
8. **Implement Data Update Logic**
    - Read the amount entered by the user from the input field.
    - Validate the input to ensure:
        - the field is not empty
        - the entered value is numeric
        - the amount is not negative
    - Get the selected day from the Spinner.

- Update the corresponding expense value in the dataset.

9. **Populate the Bar Chart**
    - Convert the expense values into **BarEntry** objects.
    - Create a **BarDataSet** and apply suitable colors and value labels.
    - Set the dataset to the **BarChart** using **BarData**.
    - Refresh the chart using `invalidate()`.
10. **Populate the Line Chart**

- Convert the expense values into **Entry** objects.
- Create a **LineDataSet** to represent the weekly expense trend.
- Customize the line chart with colors, circle markers, filled area, and value labels.
- Set the dataset to the **LineChart** using **LineData**.
- Refresh the chart using `invalidate()`.

11. **Populate the Pie Chart**

- Convert non-zero expense values into **PieEntry** objects.
- Create a **PieDataSet** to represent spending distribution.
- Apply suitable colors, slice spacing, and value text size.
- Set the dataset to the **PieChart** using **PieData**.
- If no data is available, display **No Data Available** in the chart center.
- Refresh the chart using `invalidate()`.

12. **Update Dashboard Summary**

- Calculate the **total weekly expense** by summing all values in the dataset.
- Find the **highest spending day** by identifying the maximum expense value.
- Display these details in the summary TextViews at the top of the dashboard.

13. **Provide User Feedback**

- Clear the input field after updating the data.
- Display a **Toast message** to confirm that the dashboard has been updated successfully.

14. **Test the Application**

- Run the application in the **Android Emulator** or on a physical device.
- Enter different expense values for various days.

- Verify that all three charts and summary details update correctly.

15. **Build and Run the Application**

- Clean and rebuild the project.
- Execute the application and confirm that the dashboard displays expense data clearly using charts and summary information.

AndroidManifest.xml

```
<?xml version="1.0" encoding="utf-8"?>
<manifest xmlns:android="http://schemas.android.com/apk/res/android">

    <application
        android:allowBackup="true"
        android:dataExtractionRules="@xml/data_extraction_rules"
        android:fullBackupContent="@xml/backup_rules"
        android:icon="@mipmap/ic_launcher"
        android:label="@string/app_name"
        android:roundIcon="@mipmap/ic_launcher_round"
        android:supportsRtl="true"
        android:theme="@style/Theme.DATAVISUALIZATIONAPP">
        <activity
            android:name=".MainActivity"
            android:exported="true"
            android:label="@string/app_name"
            android:theme="@style/Theme.DATAVISUALIZATIONAPP">
            <intent-filter>
                <action android:name="android.intent.action.MAIN" />

                <category android:name="android.intent.category.LAUNCHER" />
            </intent-filter>
        </activity>
    </application>

</manifest>
```

activity_main.xml

```
<?xml version="1.0" encoding="utf-8"?>
<ScrollView xmlns:android="http://schemas.android.com/apk/res/android"
    xmlns:app="http://schemas.android.com/apk/res-auto"
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    android:fillViewport="true">

    <androidx.constraintlayout.widget.ConstraintLayout
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:padding="16dp">

        <TextView
            android:id="@+id/tvTitle"
            android:layout_width="wrap_content"
            android:layout_height="wrap_content"
            android:text="Financial Dashboard"
            android:textColor="@android:color/black"
            android:textSize="24sp"
            android:textStyle="bold"
            app:layout_constraintEnd_toEndOf="parent"
            app:layout_constraintStart_toStartOf="parent"
            app:layout_constraintTop_toTopOf="parent" />

        <TextView
            android:id="@+id/tvSummary"
            android:layout_width="0dp"
            android:layout_height="wrap_content"
            android:layout_marginTop="12dp"
            android:text="Total Expense: $0.00"
            android:textColor="@android:color/black"
            android:textSize="16sp"
            android:textStyle="bold"
            app:layout_constraintEnd_toEndOf="parent"
            app:layout_constraintStart_toStartOf="parent"
            app:layout_constraintTop_toBottomOf="@id/tvTitle" />

        <TextView
            android:id="@+id/tvHighest"
            android:layout_width="0dp"
            android:layout_height="wrap_content"
            android:layout_marginTop="4dp"
            android:text="Highest Spending Day: -"
            android:textColor="@android:color/black"
            android:textSize="15sp"
            app:layout_constraintEnd_toEndOf="parent"
            app:layout_constraintStart_toStartOf="parent"
            app:layout_constraintTop_toBottomOf="@id/tvSummary" />

        <LinearLayout
            android:id="@+id/inputLayout"
            android:layout_width="0dp"
            android:layout_height="wrap_content"
            android:layout_marginTop="16dp"
```

```
        android:orientation="horizontal"
        app:layout_constraintEnd_toEndOf="parent"
        app:layout_constraintStart_toStartOf="parent"
        app:layout_constraintTop_toBottomOf="@id/tvHighest">

        <Spinner
            android:id="@+id/spinnerDays"
            android:layout_width="0dp"
            android:layout_height="48dp"
            android:layout_weight="1" />

        <com.google.android.material.textfield.TextInputLayout
            android:layout_width="0dp"
            android:layout_height="wrap_content"
            android:layout_marginStart="8dp"
            android:layout_weight="1"
            android:hint="Amount ($)">

            <com.google.android.material.textfield.TextInputEditText
                android:id="@+id/etAmount"
                android:layout_width="match_parent"
                android:layout_height="wrap_content"
                android:inputType="numberDecimal" />
        </com.google.android.material.textfield.TextInputLayout>
    </LinearLayout>

    <Button
        android:id="@+id/btnUpdate"
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:layout_marginTop="8dp"
        android:text="Update All Charts"
        app:layout_constraintTop_toBottomOf="@id/inputLayout" />

    <TextView
        android:id="@+id/labelBar"
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:layout_marginTop="24dp"
        android:text="1. Bar Chart: Daily Comparison"
        android:textStyle="bold"
        app:layout_constraintStart_toStartOf="parent"
        app:layout_constraintTop_toBottomOf="@id/btnUpdate" />

    <com.github.mikephil.charting.charts.BarChart
        android:id="@+id/barChart"
        android:layout_width="match_parent"
        android:layout_height="250dp"
        android:layout_marginTop="8dp"
        app:layout_constraintTop_toBottomOf="@id/labelBar" />

    <TextView
        android:id="@+id/labelLine"
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:layout_marginTop="24dp"
```

```
            android:text="2. Line Chart: Weekly Trend"
            android:textStyle="bold"
            app:layout_constraintStart_toStartOf="parent"
            app:layout_constraintTop_toBottomOf="@id/barChart" />

        <com.github.mikephil.charting.charts.LineChart
            android:id="@+id/lineChart"
            android:layout_width="match_parent"
            android:layout_height="250dp"
            android:layout_marginTop="8dp"
            app:layout_constraintTop_toBottomOf="@id/labelLine" />

        <TextView
            android:id="@+id/labelPie"
            android:layout_width="wrap_content"
            android:layout_height="wrap_content"
            android:layout_marginTop="24dp"
            android:text="3. Pie Chart: Spending Distribution"
            android:textStyle="bold"
            app:layout_constraintStart_toStartOf="parent"
            app:layout_constraintTop_toBottomOf="@id/lineChart" />

        <com.github.mikephil.charting.charts.PieChart
            android:id="@+id/pieChart"
            android:layout_width="match_parent"
            android:layout_height="300dp"
            android:layout_marginTop="8dp"
            android:layout_marginBottom="16dp"
            app:layout_constraintBottom_toBottomOf="parent"
            app:layout_constraintTop_toBottomOf="@id/labelPie" />

    </androidx.constraintlayout.widget.ConstraintLayout>
</ScrollView>
```

MainActivity.kt

```
package com.example.datavisualizationapp

import android.graphics.Color
import android.os.Bundle
import android.widget.ArrayAdapter
import android.widget.Toast
import androidx.appcompat.app.AppCompatActivity
import com.example.datavisualizationapp.databinding.ActivityMainBinding
import com.github.mikephil.charting.components.Legend
import com.github.mikephil.charting.components.XAxis
import com.github.mikephil.charting.data.BarData
import com.github.mikephil.charting.data.BarDataSet
import com.github.mikephil.charting.data.BarEntry
import com.github.mikephil.charting.data.Entry
import com.github.mikephil.charting.data.LineData
import com.github.mikephil.charting.data.LineDataSet
import com.github.mikephil.charting.data.PieData
import com.github.mikephil.charting.data.PieDataSet
import com.github.mikephil.charting.data.PieEntry
```

```
import com.github.mikephil.charting.formatter.IndexAxisValueFormatter
import com.github.mikephil.charting.formatter.ValueFormatter
import com.github.mikephil.charting.utils.ColorTemplate
import java.util.Locale

class MainActivity : AppCompatActivity() {

    private lateinit var binding: ActivityMainBinding

    private val labels = listOf("Mon", "Tue", "Wed", "Thu", "Fri", "Sat",
"Sun")

    // Default sample data for immediate chart display
    private val expenses = mutableListOf(120f, 180f, 90f, 250f, 300f, 140f,
200f)

    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)
        binding = ActivityMainBinding.inflate(layoutInflater)
        setContentView(binding.root)

        setupSpinner()
        setupCharts()
        updateSummary()

        binding.btnUpdate.setOnClickListener {
            updateData()
        }
    }

    private fun setupSpinner() {
        val adapter = ArrayAdapter(this, android.R.layout.simple_spinner_item,
labels)

adapter.setDropDownViewResource(android.R.layout.simple_spinner_dropdown_item)
        binding.spinnerDays.adapter = adapter
    }

    private fun updateData() {
        val amountStr = binding.etAmount.text.toString().trim()

        if (amountStr.isEmpty()) {
            Toast.makeText(this, "Please enter an amount",
Toast.LENGTH_SHORT).show()
            return
        }

        val amount = amountStr.toFloatOrNull()
        if (amount == null) {
            Toast.makeText(this, "Enter a valid amount",
Toast.LENGTH_SHORT).show()
            return
        }

        if (amount < 0f) {
            Toast.makeText(this, "Amount cannot be negative",
```

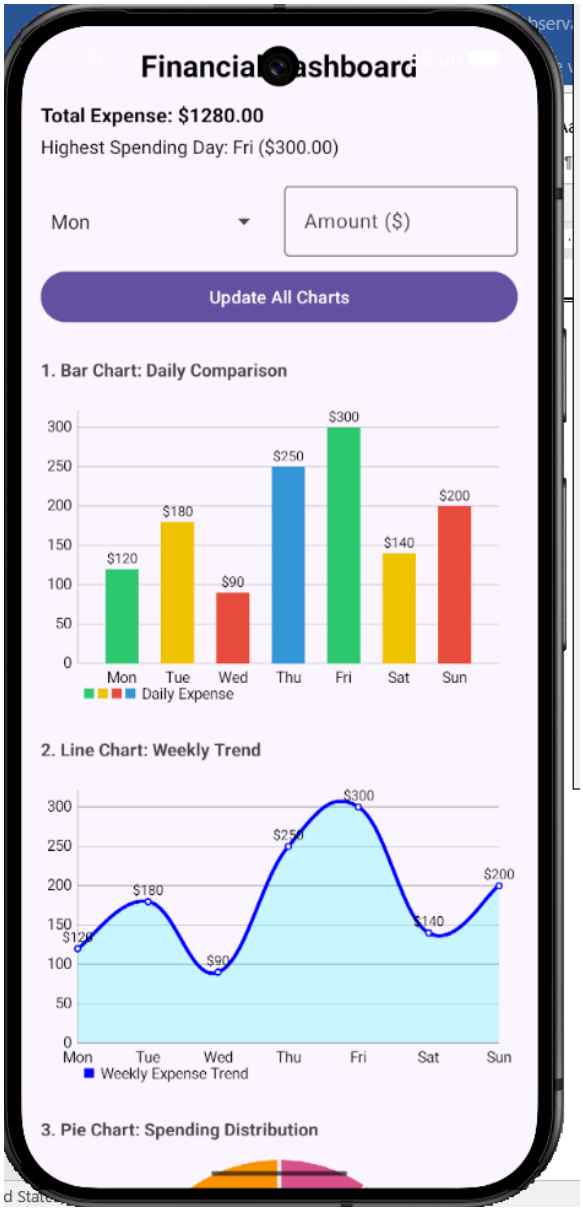
```
Toast.LENGTH_SHORT).show()
            return
        }

        val dayIndex = binding.spinnerDays.selectedItemPosition
        expenses[dayIndex] = amount

        updateCharts()
        updateSummary()

        binding.etAmount.text?.clear()
        Toast.makeText(this, "Dashboard Updated", Toast.LENGTH_SHORT).show()
    }

    private fun setupCharts() {
        val xAxisFormatter = IndexAxisValueFormatter(labels)

        // Bar Chart Setup
        binding.barChart.apply {
            description.isEnabled = false
            setFitBars(true)
            animateY(1000)
            legend.apply {
                isEnabled = true
                textSize = 12f
                form = Legend.LegendForm.SQUARE
            }
            xAxis.apply {
                valueFormatter = xAxisFormatter
                position = XAxis.XAxisPosition.BOTTOM
                setDrawGridLines(false)
                granularity = 1f
                textSize = 12f
            }
            axisRight.isEnabled = false
            axisLeft.axisMinimum = 0f
            axisLeft.textSize = 12f
        }

        // Line Chart Setup
        binding.lineChart.apply {
            description.isEnabled = false
            animateX(1000)
            legend.apply {
                isEnabled = true
                textSize = 12f
            }
            xAxis.apply {
                valueFormatter = xAxisFormatter
                position = XAxis.XAxisPosition.BOTTOM
                setDrawGridLines(false)
                granularity = 1f
                textSize = 12f
            }
            axisRight.isEnabled = false
            axisLeft.axisMinimum = 0f
```

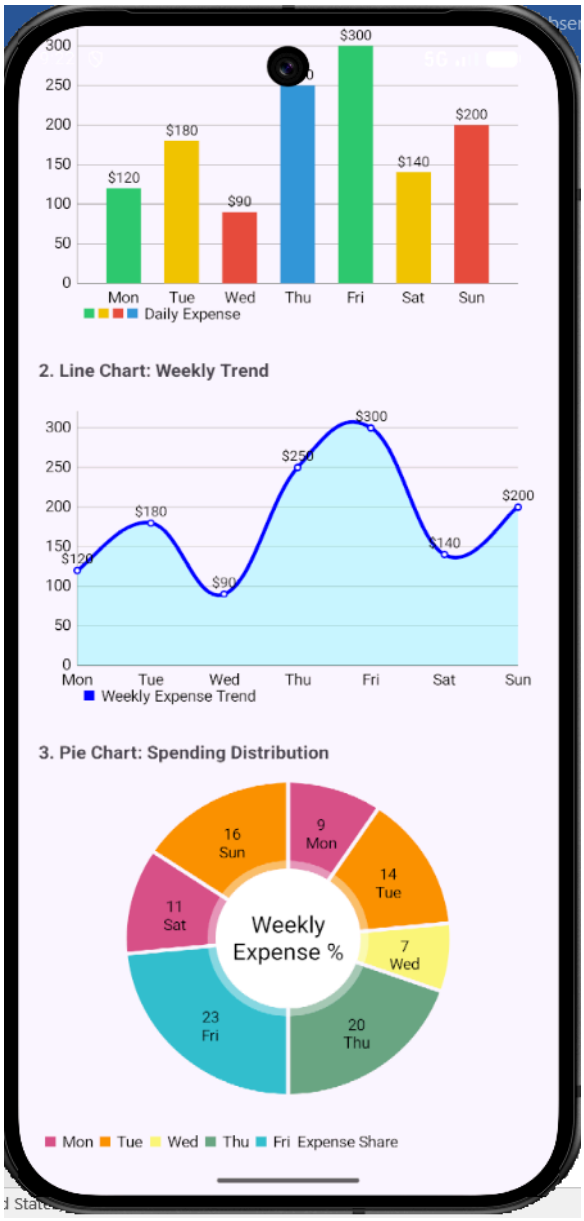
```
            axisLeft.textSize = 12f
        }

        // Pie Chart Setup
        binding.pieChart.apply {
            description.isEnabled = false
            setUsePercentValues(true)
            setEntryLabelColor(Color.BLACK)
            setEntryLabelTextSize(12f)
            animateY(1400)
            centerText = "Weekly Expense %"
            setCenterTextSize(18f)
            holeRadius = 45f
            transparentCircleRadius = 50f
            legend.apply {
                isEnabled = true
                textSize = 12f
            }
        }

        updateCharts()
    }

    private fun updateCharts() {
        updateBarChart()
        updateLineChart()
        updatePieChart()
    }

    private fun updateBarChart() {
        val barEntries = ArrayList<BarEntry>()
        for (i in expenses.indices) {
            barEntries.add(BarEntry(i.toFloat(), expenses[i]))
        }

        val barDataSet = BarDataSet(barEntries, "Daily Expense").apply {
            colors = ColorTemplate.MATERIAL_COLORS.toList()
            valueTextSize = 11f
            valueTextColor = Color.BLACK
            setDrawValues(true)
        }

        val barData = BarData(barDataSet).apply {
            barWidth = 0.6f
            setValueFormatter(CurrencyValueFormatter())
        }

        binding.barChart.data = barData
        binding.barChart.invalidate()
    }

    private fun updateLineChart() {
        val lineEntries = ArrayList<Entry>()
        for (i in expenses.indices) {
            lineEntries.add(Entry(i.toFloat(), expenses[i]))
        }
```

```
        val lineDataSet = LineDataSet(lineEntries, "Weekly Expense
Trend").apply {
            color = Color.BLUE
            setCircleColor(Color.BLUE)
            valueTextColor = Color.BLACK
            valueTextSize = 11f
            lineWidth = 3f
            mode = LineDataSet.Mode.CUBIC_BEZIER
            setDrawFilled(true)
            fillColor = Color.CYAN
            fillAlpha = 50
            setDrawValues(true)
        }

        val lineData = LineData(lineDataSet).apply {
            setValueFormatter(CurrencyValueFormatter())
        }

        binding.lineChart.data = lineData
        binding.lineChart.invalidate()
    }

    private fun updatePieChart() {
        val pieEntries = ArrayList<PieEntry>()
        for (i in expenses.indices) {
            if (expenses[i] > 0f) {
                pieEntries.add(PieEntry(expenses[i], labels[i]))
            }
        }

        if (pieEntries.isEmpty()) {
            binding.pieChart.clear()
            binding.pieChart.centerText = "No Data Available"
            binding.pieChart.invalidate()
            return
        }

        val pieDataSet = PieDataSet(pieEntries, "Expense Share").apply {
            colors = ColorTemplate.JOYFUL_COLORS.toList()
            sliceSpace = 3f
            valueTextSize = 12f
            valueTextColor = Color.BLACK
        }

        val pieData = PieData(pieDataSet)
        binding.pieChart.data = pieData
        binding.pieChart.centerText = "Weekly Expense %"
        binding.pieChart.invalidate()
    }

    private fun updateSummary() {
        val total = expenses.sum()
        val maxExpense = expenses.maxOrNull() ?: 0f
        val highestDayIndex = expenses.indexOf(maxExpense)
        val highestDay = if (highestDayIndex >= 0) labels[highestDayIndex] else
```

```
"-"

        binding.tvSummary.text = "Total Expense: ${formatCurrency(total)}"
        binding.tvHighest.text = "Highest Spending Day: $highestDay
($formatCurrency(maxExpense)})"
    }

    private fun formatCurrency(value: Float): String {
        return String.format(Locale.US, "$%.2f", value)
    }

    private class CurrencyValueFormatter : ValueFormatter() {
        override fun getFormattedValue(value: Float): String {
            return String.format(Locale.US, "$%.0f", value)
        }
    }
}
```

Output





Result

Thus, a **Data Visualization App** was successfully designed and developed to display user data using charts and graphs, providing clear and effective graphical representation of the information.

Ex. No. : 6 **Date: 11.03.26**

Register No.: 231701032 **Name: Mohamed Zaidh A**

Chat Application UI

Aim

Design the UI for a chat application with message bubbles, avatars, and a responsive input field. Design Focus: Seamless interaction, UI consistency.

Procedure

1. **Requirement Analysis**
   Identify the objective of designing a **chat application user interface** that includes message bubbles, user avatars, and an input field. The UI should support smooth interaction, clear message display, and consistent visual design.
2. **Create an Android Studio Project**
   - Open **Android Studio**.
   - Select **New Project → Empty Activity**.
   - Enter the application name as **Chat Application UI**.
   - Choose **Kotlin** as the programming language.
   - Select **Minimum SDK (API 21 or above)**.
3. **Design the Main Layout**
   - Open `activity_main.xml`.
   - Use **ConstraintLayout or LinearLayout** as the root layout.
   - Divide the screen into:
     - Chat message area (top section)
     - Input area (bottom section)
4. **Add Message Display Area**
   - Use a **RecyclerView** or **ScrollView with LinearLayout** to display messages.
   - Set vertical orientation to display messages one below the other.
   - Ensure the layout supports scrolling for long conversations.
5. **Design Message Bubble Layouts**
   - Create separate XML layouts for:
     - **Sender message bubble** (aligned to the right)
     - **Receiver message bubble** (aligned to the left)
   - Use background shapes with rounded corners to resemble chat bubbles.
   - Apply different colors for sender and receiver messages.
6. **Add User Avatars**
   - Include **ImageView** elements in the message layouts.

- Display avatars next to message bubbles.
- Position avatars:
  - Left side for received messages
  - Right side for sent messages (optional)

7. **Design the Input Section**
   - Add an **EditText or TextInputEditText** for typing messages.
   - Add a **Send Button** next to the input field.
   - Align the input section at the bottom of the screen.
   - Ensure the input field expands properly and remains responsive.
8. **Implement Message Handling Logic**
   - In `MainActivity.kt`, initialize the message list.
   - Capture user input from the input field.
   - Append new messages to the message list when the send button is clicked.
   - Clear the input field after sending a message.
9. **Bind Data to the Message View**
   - If using RecyclerView:
     - Create an adapter class.
     - Bind messages dynamically to the chat UI.
   - Differentiate between sender and receiver messages using view types.
10. **Apply UI Styling and Consistency**

- Use consistent colors, fonts, and spacing across the UI.
- Maintain uniform padding and margins for message bubbles.
- Ensure proper alignment of text and avatars.

11. **Handle Keyboard Interaction**

- Ensure the input field remains visible when the keyboard is opened.
- Adjust layout using appropriate attributes like `adjustResize`.

12. **Test the Application**

- Run the application in the **Android Emulator or on a physical device**.
- Send messages and verify:
  - Proper alignment of message bubbles
  - Smooth scrolling of chat messages
  - Correct input handling

13. **Build and Run the Application**

- Clean and rebuild the project.
- Execute the application and verify that the chat UI functions correctly.

AndroidManifest.xml

```
<?xml version="1.0" encoding="utf-8"?>
<manifest xmlns:android="http://schemas.android.com/apk/res/android"
    xmlns:tools="http://schemas.android.com/tools">

    <application
        android:allowBackup="true"
        android:dataExtractionRules="@xml/data_extraction_rules"
        android:fullBackupContent="@xml/backup_rules"
        android:icon="@mipmap/ic_launcher"
        android:label="@string/app_name"
        android:roundIcon="@mipmap/ic_launcher_round"
        android:supportsRtl="true"
        android:theme="@style/Theme.ChatApplicationUI">
        <activity
            android:name=".MainActivity"
            android:exported="true"
            android:label="@string/app_name"
            android:windowSoftInputMode="adjustResize"
            android:theme="@style/Theme.ChatApplicationUI">
            <intent-filter>
                <action android:name="android.intent.action.MAIN" />

                <category android:name="android.intent.category.LAUNCHER" />
            </intent-filter>
        </activity>
    </application>

</manifest>
```

activity_main.xml

```
<?xml version="1.0" encoding="utf-8"?>
<androidx.constraintlayout.widget.ConstraintLayout
xmlns:android="http://schemas.android.com/apk/res/android"
    xmlns:app="http://schemas.android.com/apk/res-auto"
    xmlns:tools="http://schemas.android.com/tools"
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    android:background="#ECE5DD"
    tools:context=".MainActivity">

    <com.google.android.material.appbar.AppBarLayout
        android:id="@+id/appBarLayout"
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        app:layout_constraintTop_toTopOf="parent">

        <androidx.appcompat.widget.Toolbar
            android:id="@+id/toolbar"
            android:layout_width="match_parent"
            android:layout_height="?attr/actionBarSize"
            android:background="?attr/colorPrimary"
            app:contentInsetStartWithNavigation="0dp">
```

```
            <LinearLayout
                android:layout_width="match_parent"
                android:layout_height="match_parent"
                android:gravity="center_vertical"
                android:orientation="horizontal">

                <com.google.android.material.imageview.ShapeableImageView
                    android:id="@+id/profileImage"
                    android:layout_width="40dp"
                    android:layout_height="40dp"
                    android:layout_marginStart="8dp"
                    android:padding="2dp"
                    android:src="@android:drawable/sym_def_app_icon"

app:shapeAppearanceOverlay="@style/ShapeAppearance.ChatApp.Circle"
                    app:strokeColor="@color/white"
                    app:strokeWidth="1dp" />

                <LinearLayout
                    android:layout_width="wrap_content"
                    android:layout_height="wrap_content"
                    android:layout_marginStart="12dp"
                    android:orientation="vertical">

                    <TextView
                        android:id="@+id/chatTitle"
                        android:layout_width="wrap_content"
                        android:layout_height="wrap_content"
                        android:text="John Doe"
                        android:textColor="@color/white"
                        android:textSize="18sp"
                        android:textStyle="bold" />

                    <TextView
                        android:layout_width="wrap_content"
                        android:layout_height="wrap_content"
                        android:text="Online"
                        android:textColor="#B3FFFFFF"
                        android:textSize="12sp" />
                </LinearLayout>
            </LinearLayout>
        </androidx.appcompat.widget.Toolbar>
    </com.google.android.material.appbar.AppBarLayout>

    <androidx.recyclerview.widget.RecyclerView
        android:id="@+id/rvChat"
        android:layout_width="0dp"
        android:layout_height="0dp"
        android:clipToPadding="false"
        android:padding="8dp"
        app:layout_constraintBottom_toTopOf="@+id/inputLayout"
        app:layout_constraintEnd_toEndOf="parent"
        app:layout_constraintStart_toStartOf="parent"
        app:layout_constraintTop_toBottomOf="@id/appBarLayout"
        tools:listitem="@layout/item_message_received" />
```

```
    <LinearLayout
        android:id="@+id/inputLayout"
        android:layout_width="0dp"
        android:layout_height="wrap_content"
        android:background="@color/white"
        android:elevation="8dp"
        android:gravity="center_vertical"
        android:orientation="horizontal"
        android:padding="8dp"
        app:layout_constraintBottom_toBottomOf="parent"
        app:layout_constraintEnd_toEndOf="parent"
        app:layout_constraintStart_toStartOf="parent">

        <EditText
            android:id="@+id/etMessage"
            android:layout_width="0dp"
            android:layout_height="wrap_content"
            android:layout_weight="1"
            android:background="@drawable/bg_input_field"
            android:hint="Type a message..."
            android:importantForAutofill="no"
            android:inputType="textMultiLine"
            android:maxLines="4"
            android:minHeight="48dp"
            android:paddingHorizontal="16dp"
            android:paddingVertical="8dp"
            android:textSize="16sp" />

        <com.google.android.material.floatingactionbutton.FloatingActionButton
            android:id="@+id/btnSend"
            android:layout_width="wrap_content"
            android:layout_height="wrap_content"
            android:layout_marginStart="8dp"
            android:contentDescription="Send"
            android:src="@android:drawable/ic_menu_send"
            app:fabCustomSize="48dp"
            app:tint="@color/white" />
    </LinearLayout>

</androidx.constraintlayout.widget.ConstraintLayout>
```

item_message_received

```
<?xml version="1.0" encoding="utf-8"?>
<androidx.constraintlayout.widget.ConstraintLayout
xmlns:android="http://schemas.android.com/apk/res/android"
    xmlns:app="http://schemas.android.com/apk/res-auto"
    android:layout_width="match_parent"
    android:layout_height="wrap_content"
    android:padding="4dp">

    <com.google.android.material.imageview.ShapeableImageView
        android:id="@+id/ivAvatar"
        android:layout_width="32dp"
        android:layout_height="32dp"
```

```
        android:src="@android:drawable/sym_def_app_icon"
        app:layout_constraintBottom_toBottomOf="@+id/tvMessage"
        app:layout_constraintStart_toStartOf="parent"
        app:shapeAppearanceOverlay="@style/ShapeAppearance.ChatApp.Circle" />

    <TextView
        android:id="@+id/tvMessage"
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:layout_marginStart="8dp"
        android:layout_marginEnd="64dp"
        android:background="@drawable/bg_received_bubble"
        android:elevation="1dp"
        android:maxWidth="280dp"
        android:paddingHorizontal="12dp"
        android:paddingVertical="8dp"
        android:text="I am doing great, thanks!"
        android:textColor="#000000"
        android:textSize="16sp"
        app:layout_constraintBottom_toBottomOf="parent"
        app:layout_constraintStart_toEndOf="@id/ivAvatar"
        app:layout_constraintTop_toTopOf="parent" />

    <TextView
        android:id="@+id/tvTime"
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:layout_marginEnd="4dp"
        android:layout_marginBottom="4dp"
        android:text="10:01 AM"
        android:textColor="#80000000"
        android:textSize="10sp"
        app:layout_constraintBottom_toBottomOf="@id/tvMessage"
        app:layout_constraintEnd_toEndOf="@id/tvMessage" />

</androidx.constraintlayout.widget.ConstraintLayout>
```

item_message_sent

```
<?xml version="1.0" encoding="utf-8"?>
<androidx.constraintlayout.widget.ConstraintLayout
xmlns:android="http://schemas.android.com/apk/res/android"
    xmlns:app="http://schemas.android.com/apk/res-auto"
    android:layout_width="match_parent"
    android:layout_height="wrap_content"
    android:padding="4dp">

    <TextView
        android:id="@+id/tvMessage"
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:layout_marginStart="64dp"
        android:background="@drawable/bg_sent_bubble"
        android:elevation="1dp"
        android:maxWidth="280dp"
```

```
        android:paddingHorizontal="12dp"
        android:paddingVertical="8dp"
        android:text="Hello, how are you?"
        android:textColor="#000000"
        android:textSize="16sp"
        app:layout_constraintBottom_toBottomOf="parent"
        app:layout_constraintEnd_toEndOf="parent"
        app:layout_constraintTop_toTopOf="parent" />

    <TextView
        android:id="@+id/tvTime"
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:layout_marginEnd="4dp"
        android:layout_marginBottom="4dp"
        android:text="10:00 AM"
        android:textColor="#80000000"
        android:textSize="10sp"
        app:layout_constraintBottom_toBottomOf="@id/tvMessage"
        app:layout_constraintEnd_toEndOf="@id/tvMessage" />

</androidx.constraintlayout.widget.ConstraintLayout>
```

MainActivity.kt

```
package com.example.chatapplicationui

import android.os.Bundle
import androidx.appcompat.app.AppCompatActivity
import androidx.recyclerview.widget.LinearLayoutManager
import com.example.chatapplicationui.databinding.ActivityMainBinding

class MainActivity : AppCompatActivity() {

    private lateinit var binding: ActivityMainBinding
    private lateinit var adapter: ChatAdapter
    private val messages = mutableListOf<Message>()

    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)
        binding = ActivityMainBinding.inflate(layoutInflater)
        setContentView(binding.root)

        setupRecyclerView()
        loadDummyMessages()

        binding.btnSend.setOnClickListener {
            val text = binding.etMessage.text.toString().trim()
            if (text.isNotEmpty()) {
                sendMessage(text)
            }
        }
    }

    private fun setupRecyclerView() {
```

```
        adapter = ChatAdapter(messages)
        binding.rvChat.layoutManager = LinearLayoutManager(this).apply {
            stackFromEnd = true
        }
        binding.rvChat.adapter = adapter
    }

    private fun loadDummyMessages() {
        messages.add(Message("Hey there!", "10:00 AM", false))
        messages.add(Message("Hi! How's it going?", "10:01 AM", true))
        messages.add(Message("Everything is going well. I'm working on the new
UI design.", "10:02 AM", false))
        messages.add(Message("That sounds exciting! Can't wait to see it.",
"10:03 AM", true))
        messages.add(Message("I'll share some screenshots once it's ready.",
"10:05 AM", false))
        messages.add(Message("Awesome!", "10:06 AM", true))
        adapter.notifyDataSetChanged()
        binding.rvChat.scrollToPosition(messages.size - 1)
    }

    private fun sendMessage(text: String) {
        val newMessage = Message(text, "10:07 AM", true)
        messages.add(newMessage)
        adapter.notifyItemInserted(messages.size - 1)
        binding.rvChat.scrollToPosition(messages.size - 1)
        binding.etMessage.text.clear()
    }
}
```

ChatAdapter.kt

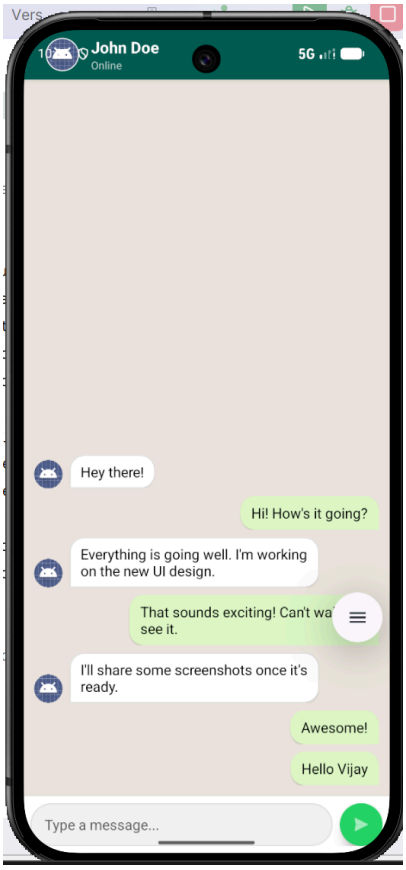
```
package com.example.chatapplicationui

import android.view.LayoutInflater
import android.view.View
import android.view.ViewGroup
import android.widget.TextView
import androidx.recyclerview.widget.RecyclerView
import com.example.chatapplicationui.databinding.ItemMessageReceivedBinding
import com.example.chatapplicationui.databinding.ItemMessageSentBinding

class ChatAdapter(private val messages: List<Message>) :
RecyclerView.Adapter<RecyclerView.ViewHolder>() {

    private val TYPE_SENT = 1
    private val TYPE_RECEIVED = 2

    override fun getItemViewType(position: Int): Int {
        return if (messages[position].isSent) TYPE_SENT else TYPE_RECEIVED
    }

    override fun onCreateViewHolder(parent: ViewGroup, viewType: Int):
RecyclerView.ViewHolder {
        return if (viewType == TYPE_SENT) {
            val binding =
```

```
ItemMessageSentBinding.inflate(LayoutInflater.from(parent.context), parent,
false)
            SentViewHolder(binding)
        } else {
            val binding =
ItemMessageReceivedBinding.inflate(LayoutInflater.from(parent.context), parent,
false)
            ReceivedViewHolder(binding)
        }
    }

    override fun onBindViewHolder(holder: RecyclerView.ViewHolder, position:
Int) {
        val message = messages[position]
        if (holder is SentViewHolder) {
            holder.bind(message)
        } else if (holder is ReceivedViewHolder) {
            holder.bind(message)
        }
    }

    override fun getItemCount(): Int = messages.size

    inner class SentViewHolder(private val binding: ItemMessageSentBinding) :
RecyclerView.ViewHolder(binding.root) {
        fun bind(message: Message) {
            binding.tvMessage.text = message.text
            binding.tvTime.text = message.time
        }
    }

    inner class ReceivedViewHolder(private val binding:
ItemMessageReceivedBinding) : RecyclerView.ViewHolder(binding.root) {
        fun bind(message: Message) {
            binding.tvMessage.text = message.text
            binding.tvTime.text = message.time
        }
    }
}
```

Message.kt

```
package com.example.chatapplicationui

data class Message(
    val text: String,
    val time: String,
    val isSent: Boolean
)
```

Output



Result

Thus, a **Chat Application UI** was successfully designed and developed with message bubbles, user avatars, and a responsive input field, ensuring seamless interaction and consistent user interface design.

Ex. No. : 7 **Date: 18.03.26**

Register No.: 231701032 **Name: Mohamed Zaidh A**

Custom Animation for User Feedback

Aim

Create a mobile app that incorporates custom animations for user actions, such as button clicks or transitions between screens. Design Focus: Animation, user feedback.

Procedure

1. **Requirement Analysis**
   Identify the objective of developing a mobile application that uses **custom animations** to provide user feedback for actions such as button clicks and screen transitions.
2. **Create an Android Studio Project**
   Open **Android Studio**, create a new **Empty Activity** project, enter the name as **Custom Animation for User Feedback**, and choose **Kotlin** as the programming language.
3. **Design the Main Screen UI**
   Open `activity_main.xml` and design a simple interface using **ConstraintLayout or LinearLayout**. Add components such as:
   - TextView for title/instruction
   - Button for animation action
   - Button for screen transition
   - ImageView or CardView for animation demonstration
4. **Create Animation Resource Files**
   Create animation XML files inside the `res/anim` folder such as:
   - `scale_animation.xml`
   - `fade_animation.xml`
   - `slide_in.xml`
   - `slide_out.xml`
5. **Implement Button Click Animation**
   In `MainActivity.kt`, apply animation to a button or image using `AnimationUtils.loadAnimation()` when the user clicks the button.
6. **Implement User Feedback Animation**
   Add animations such as:
   - Scaling effect on button press
   - Fade-in/fade-out effect
   - Rotation or translation effect
   - Toast message after animation completion

7. **Create Second Activity for Transition**
   Create a new activity named `SecondActivity`. Design `activity_second.xml` with a TextView and Button to return to the main screen.
8. **Apply Screen Transition Animation**
   Use `Intent` to move from `MainActivity` to `SecondActivity`. Apply transition animations using:

   ```
   overridePendingTransition(R.anim.slide_in, R.anim.slide_out)
   ```

9. **Handle Return Transition**
   In `SecondActivity.kt`, add a back button and apply reverse animation while returning to the main screen.
10. **Test the Application**
    Run the app in the emulator or physical device. Test:

- Button click animation
- Image/Card animation
- Screen transition animation
- User feedback messages

11. **Build and Run the Application**
    Clean and rebuild the project. Execute the application and verify that animations are smooth and provide clear feedback to the user.

AndroidManifest.xml

```
<?xml version="1.0" encoding="utf-8"?>
<manifest xmlns:android="http://schemas.android.com/apk/res/android">

    <application
        android:allowBackup="true"
        android:label="Animation App"
        android:supportsRtl="true"
        android:theme="@style/Theme.AppCompat.Light.DarkActionBar">

        <activity android:name=".SecondActivity" />

        <activity
            android:name=".MainActivity"
            android:exported="true">

            <intent-filter>
                <action android:name="android.intent.action.MAIN"/>
                <category android:name="android.intent.category.LAUNCHER"/>
            </intent-filter>

        </activity>

    </application>

</manifest>
```

activity_main.xml

```
<?xml version="1.0" encoding="utf-8"?>
<LinearLayout xmlns:android="http://schemas.android.com/apk/res/android"
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    android:orientation="vertical"
    android:gravity="center"
    android:padding="20dp">

    <TextView
        android:id="@+id/tvTitle"
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:text="Custom Animation Demo"
        android:textSize="26sp"
        android:textStyle="bold"
        android:layout_marginBottom="30dp"/>

    <ImageView
        android:id="@+id/imageView"
        android:layout_width="120dp"
        android:layout_height="120dp"
        android:src="@mipmap/ic_launcher"
        android:layout_marginBottom="30dp"/>

    <Button
```

```
        android:id="@+id/btnAnimate"
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:text="Apply Animation"/>

    <Button
        android:id="@+id/btnNext"
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:text="Go To Next Screen"
        android:layout_marginTop="15dp"/>

</LinearLayout>
```

activity_second.xml

```
<?xml version="1.0" encoding="utf-8"?>
<LinearLayout xmlns:android="http://schemas.android.com/apk/res/android"
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    android:gravity="center"
    android:orientation="vertical"
    android:padding="20dp">

    <TextView
        android:id="@+id/tvSecond"
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:text="Second Screen"
        android:textSize="28sp"
        android:textStyle="bold"
        android:layout_marginBottom="30dp"/>

    <Button
        android:id="@+id/btnBack"
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:text="Back To Main"/>

</LinearLayout>
```

MainActivity.kt

```
package com.example.customanimationforuserfeedback

import android.content.Intent
import android.os.Bundle
import android.view.animation.AnimationUtils
import android.widget.Button
import android.widget.ImageView
import androidx.appcompat.app.AppCompatActivity
```

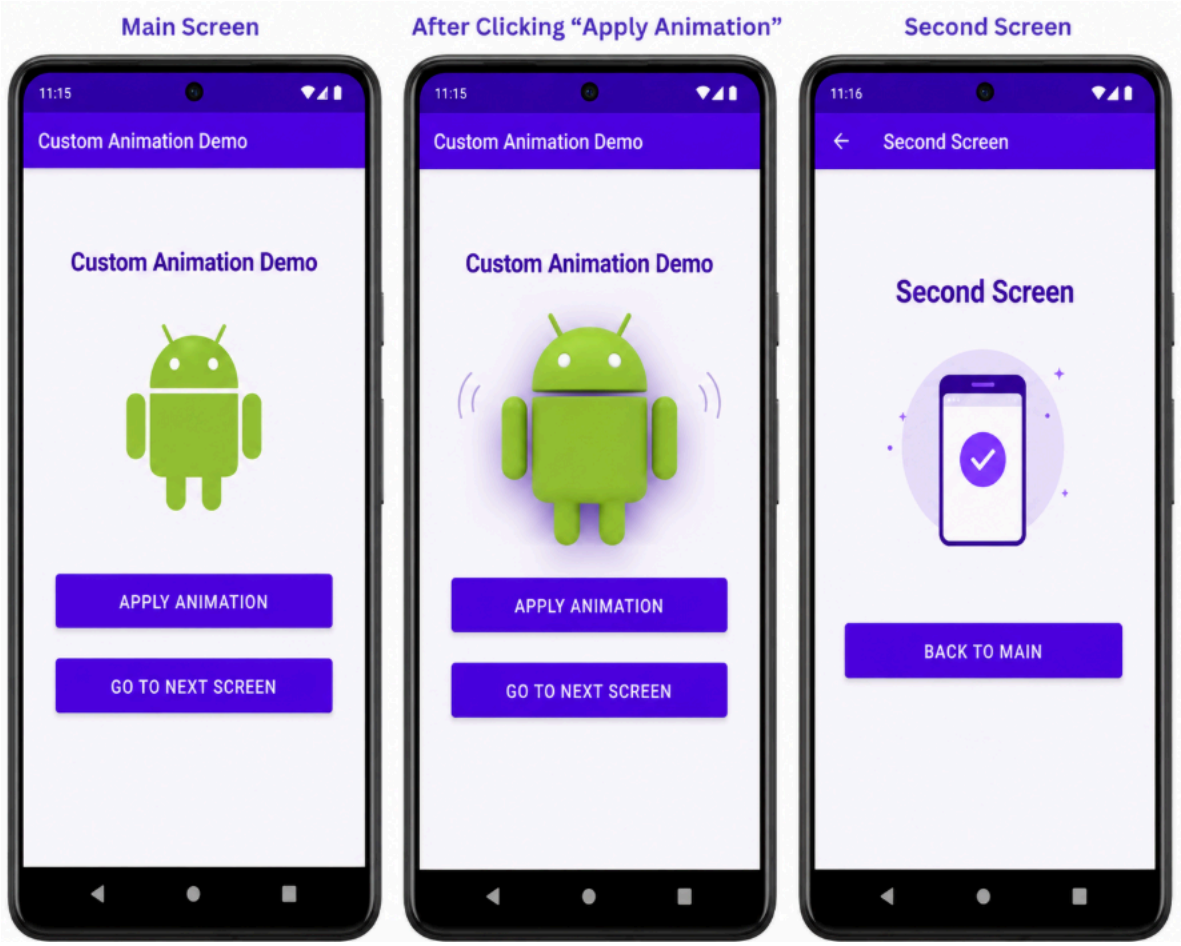
```
import com.example.exp_07.R

class MainActivity : AppCompatActivity() {

    lateinit var imageView: ImageView
    lateinit var btnAnimate: Button
    lateinit var btnNext: Button

    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)
        setContentView(R.layout.activity_main)

        imageView = findViewById(R.id.imageView)
        btnAnimate = findViewById(R.id.btnAnimate)
        btnNext = findViewById(R.id.btnNext)

        btnAnimate.setOnClickListener {

            val animation = AnimationUtils.loadAnimation(
                this,
                R.anim.bounce
            )

            imageView.startAnimation(animation)
        }

        btnNext.setOnClickListener {

            val intent = Intent(this, SecondActivity::class.java)
            startActivity(intent)
        }
    }
}
```

SecondActivity.kt

```
package com.example.customanimationforuserfeedback

import android.os.Bundle
import android.widget.Button
import androidx.appcompat.app.AppCompatActivity
import com.example.exp_07.R

class SecondActivity : AppCompatActivity() {

    lateinit var btnBack: Button

    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)
        setContentView(R.layout.activity_second)

        btnBack = findViewById(R.id.btnBack)

        btnBack.setOnClickListener {
            finish()
```

```
        }
    }
}
```

Output



Result

Thus, a mobile application was successfully developed with **custom animations for button clicks and screen transitions**, providing effective user feedback.

Ex. No. : 8 **Date: 01.04.26**

Register No.: 231701032 **Name: Mohamed Zaidh A**

Location-Based App Design

Aim

Develop a location-based app that displays the user's current location on a map with interactive features such as markers and routes. Design Focus: Location-based UI, map interactivity.

Procedure

1. **Requirement Analysis**
   Identify the objective of developing a location-based mobile application that displays the user's current location on a map and supports interactive features such as markers and routes.
2. **Create an Android Studio Project**
   Open **Android Studio**, create a new **Empty Activity** project, enter the application name as **Location-Based App Design**, and choose **Kotlin** as the programming language.
3. **Configure Google Maps API**
   - Open the Google Cloud Console.
   - Enable the **Google Maps SDK for Android**.
   - Generate an API key.
   - Add the API key inside the `AndroidManifest.xml` file.
4. **Add Required Dependencies**
   Open the `build.gradle(:app)` file and add dependencies for:
   - Google Maps
   - Location Services
   - Play Services Maps
5. **Add Permissions in AndroidManifest.xml**
   Add the following permissions:
   - `ACCESS_FINE_LOCATION`
   - `ACCESS_COARSE_LOCATION`
   - `INTERNET`
6. **Design the User Interface**
   Open `activity_main.xml` and:
   - Add a `SupportMapFragment` to display the Google Map.
   - Add optional UI components such as:
     - Buttons for location updates
     - TextViews for displaying coordinates
     - Search or route controls

7. **Initialize Google Map**
   In `MainActivity.kt`:
   - Implement `OnMapReadyCallback`.
   - Obtain the map fragment using `SupportMapFragment`.
   - Load the Google Map asynchronously using `getMapAsync()`.
8. **Enable User Location**
   - Request runtime location permissions from the user.
   - Enable the **My Location** layer on the map using:

   `googleMap.isMyLocationEnabled = true`
9. **Fetch Current Location**
   - Use `FusedLocationProviderClient` to get the user's current location.
   - Retrieve latitude and longitude values.
   - Display the location coordinates if needed.
10. **Add Marker on Map**

- Create a `LatLng` object using the current location coordinates.
- Add a marker using:

`googleMap.addMarker()`

- Set a title for the marker.

11. **Move Camera to Current Location**

- Use:

`CameraUpdateFactory.newLatLngZoom()`

- Focus the map camera on the user's current location with suitable zoom level.

12. **Implement Route or Interactive Features**

- Add multiple markers if required.
- Draw routes or paths using `PolylineOptions`.
- Enable map gestures such as zoom and drag.

13. **Test the Application**
    Run the application in an Android Emulator or physical device. Verify:

- Map loading
- Current location display
- Marker placement
- Route drawing and map interaction

14. **Handle Errors and Permissions**

- Display messages if location permission is denied.
- Handle GPS disabled or network unavailable scenarios.

15. **Build and Run the Application**
    Clean and rebuild the project. Execute the application and verify that the location-based features work correctly.

AndroidManifest.xml

```
<?xml version="1.0" encoding="utf-8"?>
<manifest xmlns:android="http://schemas.android.com/apk/res/android">

    <uses-permission android:name="android.permission.ACCESS_FINE_LOCATION" />
    <uses-permission android:name="android.permission.ACCESS_COARSE_LOCATION"
/>
```

```
    <uses-permission android:name="android.permission.INTERNET" />

    <application
        android:allowBackup="true"
        android:label="Location Based App"
        android:theme="@style/Theme.LocationBasedAppDesign">

        <meta-data
            android:name="com.google.android.geo.API_KEY"
            android:value="PASTE_YOUR_GOOGLE_MAP_API_KEY_HERE" />

        <activity
            android:name=".MainActivity"
            android:exported="true">

            <intent-filter>
                <action android:name="android.intent.action.MAIN" />
                <category android:name="android.intent.category.LAUNCHER" />
            </intent-filter>

        </activity>

    </application>

</manifest>
```

activity_main.xml

```
<?xml version="1.0" encoding="utf-8"?>
<FrameLayout xmlns:android="http://schemas.android.com/apk/res/android"
    android:layout_width="match_parent"
    android:layout_height="match_parent">

    <fragment
        android:id="@+id/mapFragment"
        android:name="com.google.android.gms.maps.SupportMapFragment"
        android:layout_width="match_parent"
        android:layout_height="match_parent" />

    <LinearLayout
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:orientation="vertical"
        android:padding="12dp"
        android:background="#CCFFFFFF">

        <TextView
            android:id="@+id/tvLocation"
            android:layout_width="match_parent"
            android:layout_height="wrap_content"
            android:text="Current Location"
            android:textSize="16sp"
            android:textStyle="bold"
            android:textColor="#000000" />
```

```
        <Button
            android:id="@+id/btnCurrentLocation"
            android:layout_width="match_parent"
            android:layout_height="wrap_content"
            android:text="Show Current Location" />

        <Button
            android:id="@+id/btnAddRoute"
            android:layout_width="match_parent"
            android:layout_height="wrap_content"
            android:text="Show Sample Route" />

    </LinearLayout>

</FrameLayout>
```

MainActivity.kt

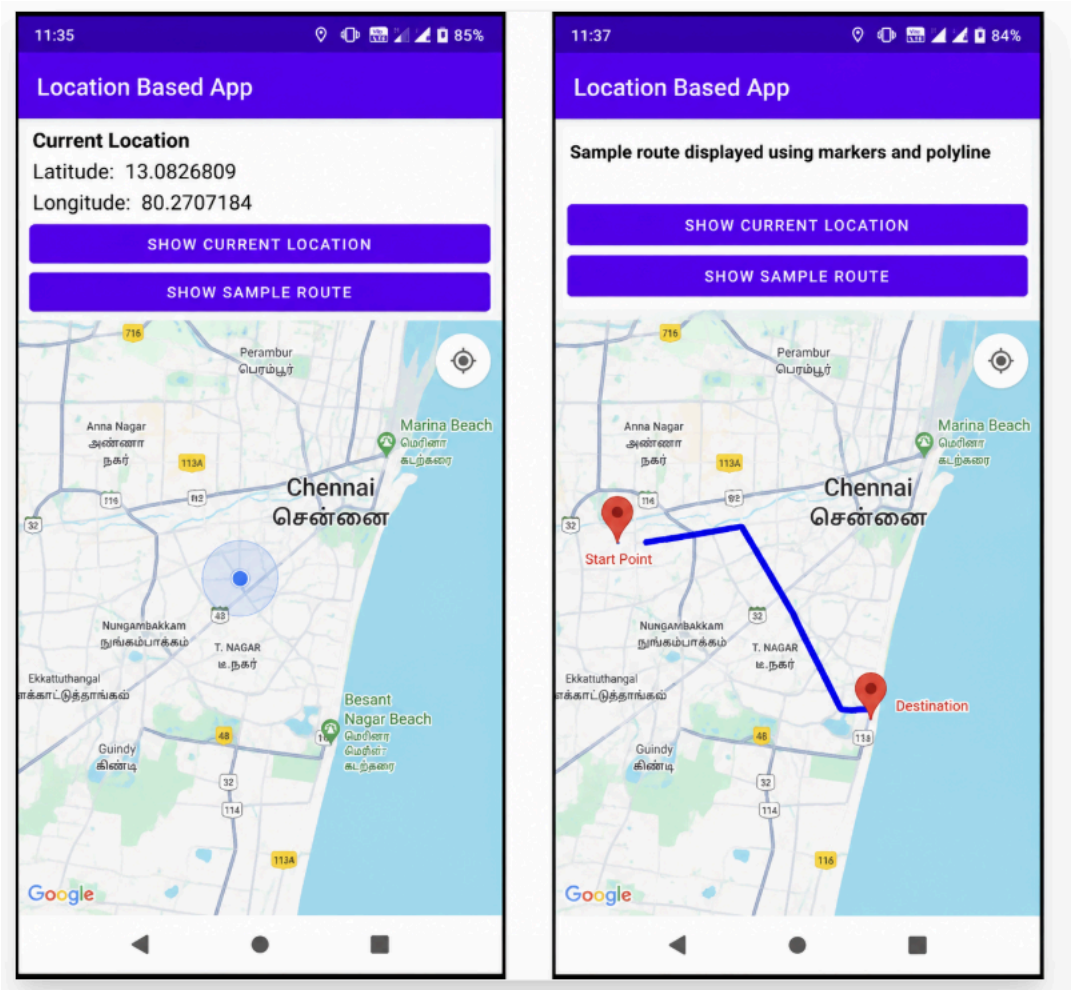
```
package com.example.exp_08

import android.Manifest
import android.content.pm.PackageManager
import android.graphics.Color
import android.os.Bundle
import android.widget.Button
import android.widget.TextView
import android.widget.Toast
import androidx.appcompat.app.AppCompatActivity
import androidx.core.app.ActivityCompat
import com.google.android.gms.location.LocationServices
import com.google.android.gms.maps.CameraUpdateFactory
import com.google.android.gms.maps.GoogleMap
import com.google.android.gms.maps.OnMapReadyCallback
import com.google.android.gms.maps.SupportMapFragment
import com.google.android.gms.maps.model.*

class MainActivity : AppCompatActivity(), OnMapReadyCallback {

    private lateinit var googleMap: GoogleMap
    private lateinit var tvLocation: TextView
    private lateinit var btnCurrentLocation: Button
    private lateinit var btnAddRoute: Button

    private val permissionCode = 100

    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)
        setContentView(R.layout.activity_main)

        tvLocation = findViewById(R.id.tvLocation)
        btnCurrentLocation = findViewById(R.id.btnCurrentLocation)
        btnAddRoute = findViewById(R.id.btnAddRoute)

        val mapFragment = supportFragmentManager
            .findFragmentById(R.id.mapFragment) as SupportMapFragment
```

```
        mapFragment.getMapAsync(this)

        btnCurrentLocation.setOnClickListener {
            getCurrentLocation()
        }

        btnAddRoute.setOnClickListener {
            drawSampleRoute()
        }
    }

    override fun onMapReady(map: GoogleMap) {
        googleMap = map

        googleMap.uiSettings.isZoomControlsEnabled = true
        googleMap.uiSettings.isCompassEnabled = true
        googleMap.uiSettings.isMapToolbarEnabled = true

        checkLocationPermission()
    }

    private fun checkLocationPermission() {
        if (ActivityCompat.checkSelfPermission(
                this,
                Manifest.permission.ACCESS_FINE_LOCATION
            ) != PackageManager.PERMISSION_GRANTED
        ) {
            ActivityCompat.requestPermissions(
                this,
                arrayOf(Manifest.permission.ACCESS_FINE_LOCATION),
                permissionCode
            )
        } else {
            enableLocation()
        }
    }

    private fun enableLocation() {
        if (ActivityCompat.checkSelfPermission(
                this,
                Manifest.permission.ACCESS_FINE_LOCATION
            ) == PackageManager.PERMISSION_GRANTED
        ) {
            googleMap.isMyLocationEnabled = true
            getCurrentLocation()
        }
    }

    private fun getCurrentLocation() {
        val fusedLocationClient =
LocationServices.getFusedLocationProviderClient(this)

        if (ActivityCompat.checkSelfPermission(
                this,
                Manifest.permission.ACCESS_FINE_LOCATION
```

```
            ) != PackageManager.PERMISSION_GRANTED
        ) {
            checkLocationPermission()
            return
        }

        fusedLocationClient.lastLocation.addOnSuccessListener { location ->
            if (location != null) {
                val currentLatLng = LatLng(location.latitude,
location.longitude)

                tvLocation.text =
                    "Latitude: ${location.latitude}\nLongitude:
${location.longitude}"

                googleMap.clear()

                googleMap.addMarker(
                    MarkerOptions()
                        .position(currentLatLng)
                        .title("You are here")
                        .snippet("Current Location")
                )

                googleMap.animateCamera(
                    CameraUpdateFactory.newLatLngZoom(currentLatLng, 16f)
                )
            } else {
                Toast.makeText(
                    this,
                    "Unable to fetch location. Enable GPS.",
                    Toast.LENGTH_SHORT
                ).show()
            }
        }
    }

    private fun drawSampleRoute() {
        googleMap.clear()

        val point1 = LatLng(13.0827, 80.2707) // Chennai
        val point2 = LatLng(13.0674, 80.2376) // Sample nearby point

        googleMap.addMarker(
            MarkerOptions()
                .position(point1)
                .title("Start Point")
        )

        googleMap.addMarker(
            MarkerOptions()
                .position(point2)
                .title("Destination")
        )

        val polylineOptions = PolylineOptions()
```

```
            .add(point1)
            .add(point2)
            .width(8f)
            .color(Color.BLUE)

        googleMap.addPolyline(polylineOptions)

        googleMap.animateCamera(
            CameraUpdateFactory.newLatLngZoom(point1, 13f)
        )

        tvLocation.text = "Sample route displayed using markers and polyline"
    }

    override fun onRequestPermissionsResult(
        requestCode: Int,
        permissions: Array<out String>,
        grantResults: IntArray
    ) {
        super.onRequestPermissionsResult(requestCode, permissions,
grantResults)

        if (requestCode == permissionCode) {
            if (grantResults.isNotEmpty() &&
                grantResults[0] == PackageManager.PERMISSION_GRANTED
            ) {
                enableLocation()
            } else {
                Toast.makeText(
                    this,
                    "Location permission denied",
                    Toast.LENGTH_SHORT
                ).show()
            }
        }
    }
}
```

Output



Result

Thus, a location-based mobile application was successfully developed to display the user's current location on a map with interactive features such as markers and routes.

Ex. No. : 9 **Date: 08.04.26**

Register No.: 231701032 **Name: Mohamed Zaidh A**

Notifications and Alerts Design

Aim

Design custom notifications and alerts, integrating both visual and auditory feedback, and aligning the notification design with the app's theme. Design Focus: Notification design, feedback.

Procedure

1. **Requirement Analysis**
   Identify the objective of developing a mobile application that displays custom notifications and alerts with visual and auditory feedback to improve user interaction.
2. **Create an Android Studio Project**
   Open **Android Studio**, create a new **Empty Activity** project, enter the application name as **Notifications and Alerts Design**, and choose **Kotlin** as the programming language.
3. **Design the User Interface**
   Open `activity_main.xml` and design the interface using **ConstraintLayout or LinearLayout**. Add UI components such as:
   - TextView for title
   - Button to trigger notification
   - Button to trigger alert dialog
   - Optional ImageView or icon for visual feedback
4. **Add Notification Permissions**
   In `AndroidManifest.xml`, add the required permission:

   ```
   <uses-permission android:name="android.permission.POST_NOTIFICATIONS"/>
   ```

5. **Create Notification Channel**
   In `MainActivity.kt`, create a notification channel for Android Oreo and above using:

   ```
   NotificationChannel
   ```

   Configure:

   - Channel ID
   - Channel name
   - Importance level
   - Description

6. **Initialize Notification Manager**
   Obtain the `NotificationManager` system service and register the notification channel.
7. **Implement Custom Notification**
   - Create a notification using `NotificationCompat.Builder`.
   - Set:
     - Small icon
     - Notification title
     - Notification message
     - Priority
     - Auto cancel option
   - Add sound, vibration, and LED feedback if required.
8. **Trigger Notification Using Button**
   Add a button click listener to display the notification using:

   ```
   notificationManager.notify()
   ```

9. **Implement Alert Dialog**
   Create a custom alert dialog using:

   ```
   AlertDialog.Builder
   ```

   Configure:

   - Title
   - Message
   - Positive and negative buttons
   - Icon if needed
10. **Add Auditory Feedback**
    Configure notification sound using default notification sound or custom sound resource.
11. **Apply Theme Consistency**
    Customize notification colors, icons, and dialog appearance to match the application theme and maintain UI consistency.
12. **Test the Application**
    Run the application in the emulator or physical device. Verify:

- Notification appearance
- Alert dialog display
- Sound and vibration feedback
- Notification tap behavior

13. **Handle Runtime Permission (Android 13+)**
    Request notification permission dynamically for Android 13 and above devices.

14. **Build and Run the Application**
    Clean and rebuild the project. Execute the application and verify that notifications and alerts function properly.

AndroidManifest.xml

```
<?xml version="1.0" encoding="utf-8"?>
<manifest xmlns:android="http://schemas.android.com/apk/res/android">

    <uses-permission android:name="android.permission.POST_NOTIFICATIONS" />
    <uses-permission android:name="android.permission.VIBRATE" />

    <application
        android:allowBackup="true"
        android:label="Notifications and Alerts"
        android:theme="@style/Theme.NotificationsAndAlertsDesign"
        android:supportsRtl="true">

        <activity
            android:name=".MainActivity"
            android:exported="true">

            <intent-filter>
                <action android:name="android.intent.action.MAIN" />
                <category android:name="android.intent.category.LAUNCHER" />
            </intent-filter>

        </activity>

    </application>

</manifest>
```

activity_main.xml

```
<?xml version="1.0" encoding="utf-8"?>
<LinearLayout xmlns:android="http://schemas.android.com/apk/res/android"
    android:id="@+id/mainLayout"
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    android:orientation="vertical"
    android:gravity="center"
    android:padding="24dp"
    android:background="#F5F5F5">

    <TextView
        android:id="@+id/tvTitle"
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:text="Notifications and Alerts Design"
        android:textSize="24sp"
        android:textStyle="bold"
        android:textAlignment="center"
        android:layout_marginBottom="30dp" />
```

```
    <Button
        android:id="@+id/btnNotification"
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:text="Show Notification" />

    <Button
        android:id="@+id/btnAlert"
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:text="Show Alert Dialog"
        android:layout_marginTop="16dp" />

    <Button
        android:id="@+id/btnToast"
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:text="Show Toast Feedback"
        android:layout_marginTop="16dp" />

</LinearLayout>
```

MainActivity.kt

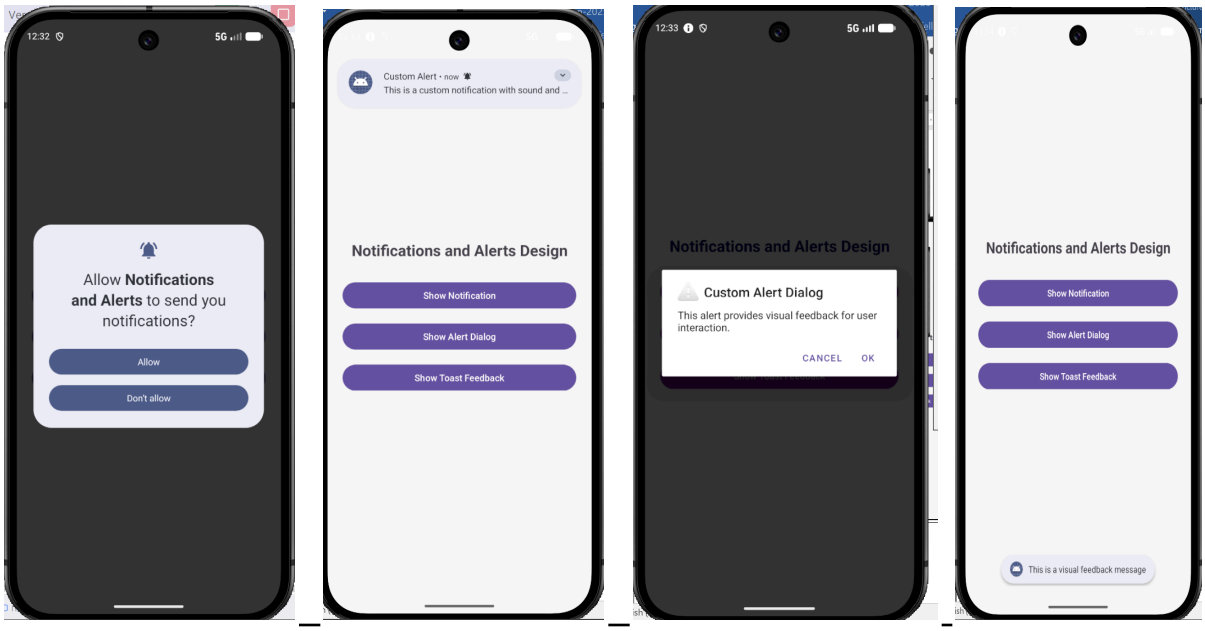
```
package com.example.exp_09

import android.Manifest
import android.app.AlertDialog
import android.app.NotificationChannel
import android.app.NotificationManager
import android.content.pm.PackageManager
import android.graphics.Color
import android.media.RingtoneManager
import android.os.Build
import android.os.Bundle
import android.os.VibrationEffect
import android.os.Vibrator
import android.os.VibratorManager
import android.widget.Button
import android.widget.Toast
import androidx.appcompat.app.AppCompatActivity
import androidx.core.app.ActivityCompat
import androidx.core.app.NotificationCompat
import androidx.core.app.NotificationManagerCompat

class MainActivity : AppCompatActivity() {

    private lateinit var btnNotification: Button
    private lateinit var btnAlert: Button
    private lateinit var btnToast: Button

    private val channelId = "alert_notification_channel"
    private val notificationId = 101
    private val permissionRequestCode = 200
```

```
    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)
        setContentView(R.layout.activity_main)

        btnNotification = findViewById(R.id.btnNotification)
        btnAlert = findViewById(R.id.btnAlert)
        btnToast = findViewById(R.id.btnToast)

        createNotificationChannel()
        requestNotificationPermission()

        btnNotification.setOnClickListener {
            showNotification()
            vibratePhone()
        }

        btnAlert.setOnClickListener {
            showAlertDialog()
        }

        btnToast.setOnClickListener {
            Toast.makeText(this, "This is a visual feedback message",
Toast.LENGTH_SHORT).show()
            vibratePhone()
        }
    }

    private fun requestNotificationPermission() {
        if (Build.VERSION.SDK_INT >= Build.VERSION_CODES.TIRAMISU) {
            if (ActivityCompat.checkSelfPermission(
                    this,
                    Manifest.permission.POST_NOTIFICATIONS
                ) != PackageManager.PERMISSION_GRANTED
            ) {
                ActivityCompat.requestPermissions(
                    this,
                    arrayOf(Manifest.permission.POST_NOTIFICATIONS),
                    permissionRequestCode
                )
            }
        }
    }

    private fun createNotificationChannel() {
        if (Build.VERSION.SDK_INT >= Build.VERSION_CODES.O) {
            val channelName = "Alert Notifications"
            val descriptionText = "Channel for custom notification alerts"
            val importance = NotificationManager.IMPORTANCE_HIGH

            val channel = NotificationChannel(channelId, channelName,
importance).apply {
                description = descriptionText
                enableLights(true)
                lightColor = Color.BLUE
                enableVibration(true)
```

```
            }

            val notificationManager =
                getSystemService(NotificationManager::class.java)

            notificationManager.createNotificationChannel(channel)
        }
    }

    private fun showNotification() {
        if (Build.VERSION.SDK_INT >= Build.VERSION_CODES.TIRAMISU) {
            if (ActivityCompat.checkSelfPermission(
                    this,
                    Manifest.permission.POST_NOTIFICATIONS
                ) != PackageManager.PERMISSION_GRANTED
            ) {
                Toast.makeText(this, "Notification permission required",
Toast.LENGTH_SHORT).show()
                requestNotificationPermission()
                return
            }
        }

        val soundUri =
RingtoneManager.getDefaultUri(RingtoneManager.TYPE_NOTIFICATION)

        val notification = NotificationCompat.Builder(this, channelId)
            .setSmallIcon(android.R.drawable.ic_dialog_info)
            .setContentTitle("Custom Alert")
            .setContentText("This is a custom notification with sound and
vibration feedback.")
            .setStyle(
                NotificationCompat.BigTextStyle()
                    .bigText("This custom notification demonstrates visual,
auditory, and vibration feedback aligned with the app theme.")
            )
            .setPriority(NotificationCompat.PRIORITY_HIGH)
            .setColor(Color.BLUE)
            .setSound(soundUri)
            .setVibrate(longArrayOf(0, 300, 200, 300))
            .setAutoCancel(true)
            .build()

        NotificationManagerCompat.from(this).notify(notificationId,
notification)
    }

    private fun showAlertDialog() {
        AlertDialog.Builder(this)
            .setTitle("Custom Alert Dialog")
            .setMessage("This alert provides visual feedback for user
interaction.")
            .setIcon(android.R.drawable.ic_dialog_alert)
            .setPositiveButton("OK") { dialog, _ ->
                Toast.makeText(this, "Alert confirmed",
Toast.LENGTH_SHORT).show()
```

```
                vibratePhone()
                dialog.dismiss()
            }
            .setNegativeButton("Cancel") { dialog, _ ->
                Toast.makeText(this, "Alert cancelled",
Toast.LENGTH_SHORT).show()
                dialog.dismiss()
            }
            .show()
    }

    private fun vibratePhone() {
        try {
            if (Build.VERSION.SDK_INT >= Build.VERSION_CODES.S) {
                val vibratorManager =
                    getSystemService(VIBRATOR_MANAGER_SERVICE) as
VibratorManager
                val vibrator = vibratorManager.defaultVibrator
                vibrator.vibrate(
                    VibrationEffect.createOneShot(
                        200,
                        VibrationEffect.DEFAULT_AMPLITUDE
                    )
                )
            } else {
                @Suppress("DEPRECATION")
                val vibrator = getSystemService(VIBRATOR_SERVICE) as Vibrator
                if (Build.VERSION.SDK_INT >= Build.VERSION_CODES.O) {
                    vibrator.vibrate(
                        VibrationEffect.createOneShot(
                            200,
                            VibrationEffect.DEFAULT_AMPLITUDE
                        )
                    )
                } else {
                    @Suppress("DEPRECATION")
                    vibrator.vibrate(200)
                }
            }
        } catch (e: Exception) {
            Toast.makeText(this, "Vibration not supported",
Toast.LENGTH_SHORT).show()
        }
    }
}
```

Output



Result

Thus, a mobile application was successfully developed to demonstrate custom notifications and alerts with visual and auditory feedback aligned to the application theme.

Ex. No. : 10 **Date: 22.04.26**

Register No.: 231701032 **Name: Mohamed Zaidh A**

Camera and Image Editing App

Aim

Create an app that captures images, applies filters, crops, and edits photos with a visually appealing UI. Design Focus: Image editing tools, intuitive interface.

Procedure

1. **Requirement Analysis**
   Identify the objective of developing a mobile application that captures images using the device camera and provides image editing features such as filters, cropping, rotation, and brightness adjustment.
2. **Create an Android Studio Project**
   Open **Android Studio**, create a new **Empty Activity** project, enter the application name as **Camera and Image Editing App**, and choose **Kotlin** as the programming language.
3. **Add Required Permissions**
   Open `AndroidManifest.xml` and add permissions for:
   - Camera access
   - Reading media/images
   - Internet (optional)
4. **Design the User Interface**
   Open `activity_main.xml` and design the UI using **ConstraintLayout or LinearLayout**. Add:
   - ImageView to display the selected image
   - Button to open the camera
   - Button to select image from gallery
   - Buttons for editing actions such as:
     - Rotate
     - Apply filter
     - Crop
     - Reset image
5. **Implement Camera Capture Functionality**
   - Use `Intent(MediaStore.ACTION_IMAGE_CAPTURE)` to open the device camera.
   - Capture the image and display it inside the ImageView.
6. **Implement Gallery Selection Functionality**
   - Use an intent to open the gallery and select an image.

    - Retrieve the selected image URI and display it in the ImageView.
7. **Handle Runtime Permissions**
    - Request camera and storage permissions dynamically at runtime.
    - Handle permission approval and denial properly.
8. **Implement Image Rotation**
    - Use ImageView rotation or Bitmap transformation methods.
    - Rotate the image by a fixed angle such as 90 degrees on button click.
9. **Implement Image Filters**
    - Apply visual effects such as:
        - Grayscale
        - Sepia
        - Brightness adjustment
        - Color filters
    - Use `ColorMatrix` or image processing methods.
10. **Implement Crop or Resize Feature**

- Allow the image to be cropped or resized programmatically.
- Update the edited image in the ImageView.

11. **Implement Reset Functionality**

- Restore the original image when the reset button is clicked.

12. **Provide User Feedback**

- Display Toast messages for successful image capture, filter application, or reset actions.
- Add smooth animations or transitions if required.

13. **Test the Application**
    Run the application in the emulator or physical device. Verify:

- Camera capture functionality
- Gallery image selection
- Image editing operations
- UI responsiveness and clarity

14. **Build and Run the Application**
    Clean and rebuild the project. Execute the application and verify that all image editing features work correctly.

AndroidManifest.xml

```
<?xml version="1.0" encoding="utf-8"?>
<manifest xmlns:android="http://schemas.android.com/apk/res/android"
    xmlns:tools="http://schemas.android.com/tools">
    <uses-permission android:name="android.permission.CAMERA" />
    <application
        android:allowBackup="true"
        android:dataExtractionRules="@xml/data_extraction_rules"
        android:fullBackupContent="@xml/backup_rules"
        android:icon="@mipmap/ic_launcher"
        android:label="@string/app_name"
        android:roundIcon="@mipmap/ic_launcher_round"
        android:supportsRtl="true"
        android:theme="@style/Theme.Exp10">
        <activity
            android:name=".MainActivity"
            android:exported="true">
            <intent-filter>
                <action android:name="android.intent.action.MAIN" />

                <category android:name="android.intent.category.LAUNCHER" />
            </intent-filter>
        </activity>
    </application>

</manifest>
```

activity_main.xml

```
<?xml version="1.0" encoding="utf-8"?>
<ScrollView xmlns:android="http://schemas.android.com/apk/res/android"
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    android:fillViewport="true">

    <LinearLayout
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:orientation="vertical"
        android:padding="16dp"
        android:gravity="center_horizontal">

        <TextView
            android:layout_width="wrap_content"
            android:layout_height="wrap_content"
            android:text="Camera and Image Editing App"
            android:textSize="24sp"
            android:textStyle="bold"
            android:layout_marginBottom="16dp" />

        <ImageView
            android:id="@+id/imageView"
            android:layout_width="match_parent"
            android:layout_height="350dp"
```

```
            android:scaleType="centerCrop"
            android:background="#E0E0E0"
            android:src="@mipmap/ic_launcher" />

        <Button
            android:id="@+id/btnCamera"
            android:layout_width="match_parent"
            android:layout_height="wrap_content"
            android:text="Capture Image"
            android:layout_marginTop="16dp" />

        <Button
            android:id="@+id/btnGallery"
            android:layout_width="match_parent"
            android:layout_height="wrap_content"
            android:text="Select from Gallery" />

        <Button
            android:id="@+id/btnRotate"
            android:layout_width="match_parent"
            android:layout_height="wrap_content"
            android:text="Rotate Image" />

        <Button
            android:id="@+id/btnGray"
            android:layout_width="match_parent"
            android:layout_height="wrap_content"
            android:text="Apply Grayscale Filter" />

        <Button
            android:id="@+id/btnCrop"
            android:layout_width="match_parent"
            android:layout_height="wrap_content"
            android:text="Crop Image" />

        <Button
            android:id="@+id/btnReset"
            android:layout_width="match_parent"
            android:layout_height="wrap_content"
            android:text="Reset Image" />

    </LinearLayout>
</ScrollView>
```

MainActivity.kt

```
package com.example.exp_10

import android.Manifest
import android.content.pm.PackageManager
import android.graphics.*
import android.net.Uri
import android.os.Bundle
import android.provider.MediaStore
import android.widget.Button
import android.widget.ImageView
import android.widget.Toast
import androidx.activity.result.contract.ActivityResultContracts
import androidx.appcompat.app.AppCompatActivity
import androidx.core.app.ActivityCompat
import androidx.core.content.ContextCompat

class MainActivity : AppCompatActivity() {

    private lateinit var imageView: ImageView

    private var currentBitmap: Bitmap? = null
    private var originalBitmap: Bitmap? = null

    private val cameraPermissionCode = 100

    private val cameraLauncher =
        registerForActivityResult(ActivityResultContracts.TakePicturePreview())
{ bitmap ->
            if (bitmap != null) {
                currentBitmap = bitmap
                originalBitmap = bitmap.copy(bitmap.config ?:
Bitmap.Config.ARGB_8888, true)
                imageView.setImageBitmap(currentBitmap)
                Toast.makeText(this, "Image captured successfully",
Toast.LENGTH_SHORT).show()
            } else {
                Toast.makeText(this, "Image capture cancelled",
Toast.LENGTH_SHORT).show()
            }
        }

    private val galleryLauncher =
        registerForActivityResult(ActivityResultContracts.GetContent()) { uri:
Uri? ->
            if (uri != null) {
                val bitmap = MediaStore.Images.Media.getBitmap(contentResolver,
uri)
                currentBitmap = bitmap
                originalBitmap = bitmap.copy(bitmap.config ?:
Bitmap.Config.ARGB_8888, true)
                imageView.setImageBitmap(currentBitmap)
                Toast.makeText(this, "Image selected from gallery",
Toast.LENGTH_SHORT).show()
            } else {
```

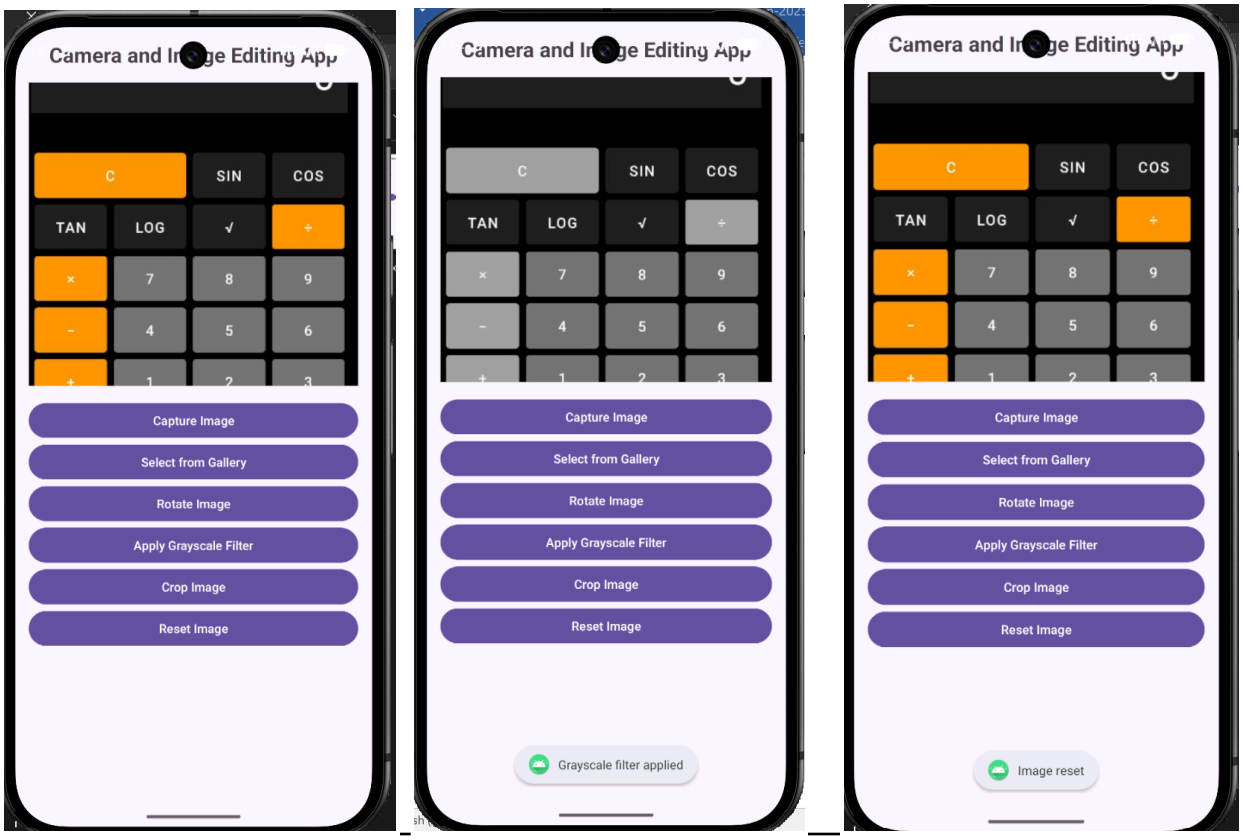
```
                Toast.makeText(this, "No image selected",
Toast.LENGTH_SHORT).show()
            }
        }

    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)
        setContentView(R.layout.activity_main)

        imageView = findViewById(R.id.imageView)

        val btnCamera: Button = findViewById(R.id.btnCamera)
        val btnGallery: Button = findViewById(R.id.btnGallery)
        val btnRotate: Button = findViewById(R.id.btnRotate)
        val btnGray: Button = findViewById(R.id.btnGray)
        val btnCrop: Button = findViewById(R.id.btnCrop)
        val btnReset: Button = findViewById(R.id.btnReset)

        btnCamera.setOnClickListener {
            checkCameraPermission()
        }

        btnGallery.setOnClickListener {
            galleryLauncher.launch("image/*")
        }

        btnRotate.setOnClickListener {
            rotateImage()
        }

        btnGray.setOnClickListener {
            applyGrayScale()
        }

        btnCrop.setOnClickListener {
            cropImage()
        }

        btnReset.setOnClickListener {
            resetImage()
        }
    }

    private fun checkCameraPermission() {
        if (ContextCompat.checkSelfPermission(this, Manifest.permission.CAMERA)
            == PackageManager.PERMISSION_GRANTED
        ) {
            cameraLauncher.launch(null)
        } else {
            ActivityCompat.requestPermissions(
                this,
                arrayOf(Manifest.permission.CAMERA),
                cameraPermissionCode
            )
        }
    }
```

```
    private fun rotateImage() {
        val bitmap = currentBitmap

        if (bitmap == null) {
            Toast.makeText(this, "Please capture or select an image first",
Toast.LENGTH_SHORT).show()
            return
        }

        val matrix = Matrix()
        matrix.postRotate(90f)

        val rotatedBitmap = Bitmap.createBitmap(
            bitmap,
            0,
            0,
            bitmap.width,
            bitmap.height,
            matrix,
            true
        )

        currentBitmap = rotatedBitmap
        imageView.setImageBitmap(currentBitmap)

        Toast.makeText(this, "Image rotated", Toast.LENGTH_SHORT).show()
    }

    private fun applyGrayScale() {
        val bitmap = currentBitmap

        if (bitmap == null) {
            Toast.makeText(this, "Please capture or select an image first",
Toast.LENGTH_SHORT).show()
            return
        }

        val grayBitmap = Bitmap.createBitmap(bitmap.width, bitmap.height,
Bitmap.Config.ARGB_8888)

        val canvas = Canvas(grayBitmap)
        val paint = Paint()
        val colorMatrix = ColorMatrix()
        colorMatrix.setSaturation(0f)

        val filter = ColorMatrixColorFilter(colorMatrix)
        paint.colorFilter = filter

        canvas.drawBitmap(bitmap, 0f, 0f, paint)

        currentBitmap = grayBitmap
        imageView.setImageBitmap(currentBitmap)

        Toast.makeText(this, "Grayscale filter applied",
Toast.LENGTH_SHORT).show()
```

```
    }

    private fun cropImage() {
        val bitmap = currentBitmap

        if (bitmap == null) {
            Toast.makeText(this, "Please capture or select an image first",
Toast.LENGTH_SHORT).show()
            return
        }

        val size = minOf(bitmap.width, bitmap.height)
        val x = (bitmap.width - size) / 2
        val y = (bitmap.height - size) / 2

        val croppedBitmap = Bitmap.createBitmap(bitmap, x, y, size, size)

        currentBitmap = croppedBitmap
        imageView.setImageBitmap(currentBitmap)

        Toast.makeText(this, "Image cropped", Toast.LENGTH_SHORT).show()
    }

    private fun resetImage() {
        if (originalBitmap == null) {
            Toast.makeText(this, "No original image available",
Toast.LENGTH_SHORT).show()
            return
        }

        currentBitmap = originalBitmap!!.copy(
            originalBitmap!!.config ?: Bitmap.Config.ARGB_8888,
            true
        )

        imageView.setImageBitmap(currentBitmap)
        Toast.makeText(this, "Image reset", Toast.LENGTH_SHORT).show()
    }

    override fun onRequestPermissionsResult(
        requestCode: Int,
        permissions: Array<out String>,
        grantResults: IntArray
    ) {
        super.onRequestPermissionsResult(requestCode, permissions,
grantResults)

        if (requestCode == cameraPermissionCode) {
            if (grantResults.isNotEmpty() &&
                grantResults[0] == PackageManager.PERMISSION_GRANTED
            ) {
                cameraLauncher.launch(null)
            } else {
                Toast.makeText(this, "Camera permission denied",
Toast.LENGTH_SHORT).show()
            }
```

```
            }
        }
}
```

Output



Result

Thus, a mobile application was successfully developed to capture and edit images with features such as filters, rotation, cropping, and reset operations using an intuitive user interface.